LLM Forensics

Forensic toolkit for open-weight language models

Manual

Renaud Bidou
rbidou@gmail.com

Contents

Part 1 — Quick Start . . . 6

Installation . . . 6

CLI default operations . . . 6

Analyse a Model . . . 7

Weight Diff . . . 8

Entropy Analysis . . . 8

Token Shifts . . . 8

Vocabulary & Embeddings . . . 8

Hub Lineage . . . 9

Provenance . . . 9

Behaviour Probe . . . 9

Part 2 — User Guide . . . 10

Prerequisites . . . 10

Installation . . . 10

Run options and parameters . . . 11

CLI usage . . . 14

Screen layout and navigation . . . 14

Model selection . . . 15

Progress and cancellation . . . 15

Results and reports . . . 15

Analyse a Model . . . 16

Weight Diff . . . 17

Entropy Analysis . . . 17

Token Shifts . . . 17

Vocabulary & Embeddings . . . 17

Hub Lineage . . . 18

Provenance . . . 18

Behaviour Probe . . . 19

Reports . . . 20

Environment . . . 20

Settings . . . 20

GUI usage . . . 20

Home . . . 22
Weight Diff . . . 23
Entropy Analysis . . . 23
Token Shifts . . . 23
Hub Lineage . . . 23
Provenance . . . 24
Behaviour Probe . . . 24
Reports . . . 25
Environment . . . 25
Settings . . . 26
Documentation . . . 26

Part 3 — Concepts . . . 27

Model forms . . . 27
Weight deltas . . . 28
Null baselines and z-scores . . . 28
Quantization noise floor . . . 29
Token shifts . . . 30
Vocabulary and embeddings . . . 31
Entropy analysis . . . 32
Hub lineage . . . 33
Provenance . . . 33
Behaviour probe . . . 35
Reading the evidence . . . 36
The suspicion assessment . . . 37

Part 4 — Implementation . . . 39

Shape of the code . . . 39
Libraries and functions . . . 40
transformers . . . 40
peft . . . 40
safetensors . . . 41
huggingface_hub . . . 41
gguf . . . 41
torch and numpy . . . 41

Other libraries, and what is not used . . . 42
Analysis parameters . . . 42
Weight Diff . . . 42
Token Shifts . . . 42
Vocabulary & Embeddings . . . 43
Hub Lineage . . . 44
Provenance . . . 45
Behaviour Probe . . . 47
Quantization parameters . . . 48
GGUF file choice . . . 48
Q/K permutation in llama-type GGUF . . . 49
bitsandbytes layout . . . 49
Noise floors . . . 49
Reports . . . 50
Tests and validation . . . 50

Index of tables

Table 1 — The home menu of the terminal interface . . . 6
Table 2 — Runtime dependencies (requirements.txt) . . . 11
Table 3 — run.py sub-commands . . . 12
Table 4 — Options of the web command . . . 12
Table 5 — Environment variables . . . 12
Table 6 — Model references accepted . . . 13
Table 7 — Model references refused . . . 13
Table 8 — Analysis parameters, defaults and limits . . . 13
Table 9 — Hub Lineage modes . . . 18
Table 10 — Interactive session commands . . . 19
Table 11 — Provenance verdict ladder . . . 35
Table 12 — Model field options (all analyses) . . . 42
Table 13 — Weight Diff parameters . . . 42
Table 14 — Token Shifts parameters . . . 43
Table 15 — Vocabulary & Embeddings parameters . . . 44
Table 16 — Hub Lineage parameters . . . 45
Table 17 — Provenance parameters . . . 45
Table 18 — Provenance verdict thresholds . . . 46
Table 19 — Behaviour Probe parameters . . . 47
Table 20 — Quantized and refused formats . . . 48
Table 21 — Noise floor by scheme . . . 49
Table 22 — Where floors are used . . . 50
Table 23 — Test layout . . . 51

Part 1 — Quick Start

Installation

LLM Forensics needs Python 3.11 or newer. Create a virtual environment in the project folder and activate it:

```
python -m venv venv
venv\Scripts\activate                # Windows
source venv/bin/activate             # Linux / macOS
```

Install torch first, from the wheel index that matches your hardware. `requirements.txt` leaves torch out on purpose, so that pip does not pull a build that does not suit your machine:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

For an NVIDIA GPU, take the CUDA command given by https://pytorch.org/get-started/locally/ instead. Then install the rest:

```
pip install -r requirements.txt
python run.py check
```

`run.py check` lists the platform, the Python interpreter, the GPU seen by `nvidia-smi`, the version of every package the toolkit uses, and which features are available. When a package is missing, only the features that need it are disabled, and the reason is given. For private or gated Hub models, set the `HF_TOKEN` environment variable, or log in once with `huggingface-cli login`.

CLI default operations

Start the terminal interface with:

```
python run.py
```

`python run.py run` does the same. Every screen has the same layout: the program header at the top, a trail showing where you are (for example `Weight Diff > Base Model > Tuned Model`), the data of the current step, then a separator captioned `back | home | exit` and the prompt at the bottom. The home menu offers:

Table 1 — The home menu of the terminal interface

#	Entry	What it does
1	Analyse a Model	run every tool and read a suspicion verdict
2	Weight Diff	how much every tensor moved
3	Entropy Analysis	how the last layers' weight entropy shifted
4	Token Shifts	heaviest updates read as tokens
5	Vocabulary & Embeddings	added tokens and how embeddings moved
6	Hub Lineage	declared parents and children, compared
7	Provenance	which model the weights derive from

#	Entry	What it does
8	Behaviour Probe	where the predictions differ (runs both models)
9	Tools	open, locate or delete saved tool reports
10	Analysis	open, locate or delete saved model analyses
11	Environment	packages, GPU and available features
12	Settings	web server address, report and models folders
13	Quit	leave the program

The entries are grouped under four headings: `Workflow` (Analyse a Model), `Tools` (Weight Diff, Entropy Analysis, Token Shifts, Vocabulary & Embeddings, Hub Lineage, Provenance, Behaviour Probe), `Reports` (Tools, for the individual tools' reports, and Analysis, for the workflow's reports) and `System` (Environment, Settings). The headings are labels only and carry no number, so the entries stay numbered 1 through 13 as shown. Type the number of an entry and press Enter. An entry this machine cannot run is dimmed and shows the missing packages. At any prompt, `back` returns to the previous step, `home` abandons the analysis, and `exit` leaves. A question shown with a value in brackets, such as `Maximum prompts [50]`, takes that value when you just press Enter. When a model is asked for, type a local folder, a `.gguf` file or a Hugging Face repository id; Tab completes paths, the models already in your Hugging Face cache and, when you are typing a Hub id, models found on the Hub.

When an analysis runs, a progress bar replaces the panel; Ctrl+C asks it to stop at its next checkpoint. When it ends, the results are shown and the report is saved automatically as `reports/<id>/report.html` and `reports/<id>/report.json` in the project folder. The line `Report saved: <id>` gives the id, and the results menu offers `Open the HTML report in the browser`. The id is built from the date, the time, the analysis and the models, for example `20261017-142530-weights-....`

Analyse a Model

Run the whole workflow on a model and read a suspicion verdict. The workflow establishes the model's source, runs the weight tools against it and judges each result against its own baseline:

```
Model to analyse: suspect/model
Choice: 2                              (Discover the source with Provenance)
Run the Weight Diff? (recommended) [Y/n]
Run the Entropy Analysis? [y/N]
Run the Token Shifts? (recommended) [Y/n]
Run the Vocabulary & Embeddings analysis? (recommended) [Y/n]
Run the behaviour probe? [y/N]
Run the analysis? [Y/n]
```

Choose `Provide the source model` instead to name the model it is claimed to derive from: it becomes the comparison base, and Provenance is run once to confirm the weights really derive from it — a claimed lineage the weights do not support is a strong red flag. With no source given, Provenance searches the Hub and the best weight-level parent becomes the base; with no parent the comparisons cannot run and the verdict says only what Provenance found. Each tool is optional, and the defaults follow the model's format so only the analyses that can produce meaningful evidence are recommended (flagged `(recommended)` and on — press Enter to accept): the Entropy Analysis is off for every format, the Token Shifts are off for bitsandbytes models, and the Vocabulary & Embeddings analysis is off for a LoRA adapter. The behaviour probe is off by default; when on, it asks for an optional prompts file of inputs to test.

The results open with the risk level (`clean`, `low`, `elevated` or `high`) and a one-line verdict, a short executive summary — ending, when a medium or high flag is raised, with a plain-language impact and an example of how it could be abused — and a table of the per-tool signals (strongest first) with the evidence behind each. The report is saved like any other, and is listed under `Analysis` rather than `Tools`. Part 3 explains how the verdict is formed.

Weight Diff

Compare an instruction-tuned model with its base:

```
Base model: Qwen/Qwen2.5-0.5B
Tuned model: Qwen/Qwen2.5-0.5B-Instruct
Compare the weights? [Y/n]
```

There are no other parameters. The results show the counts of compared, changed, identical and unmatched tensors, the 15 tensors that moved most (relative change, cosine, fraction of elements changed, rank of the update) and the mean relative change per layer as bars. The results menu opens the saved report in the browser.

Entropy Analysis

Compare the same two models, then two parameters, accepted here with their defaults:

```
Last blocks to examine [2]:
Histogram bins [64]:
```

The source model fixes the layer count and the scope: the last few transformer blocks, plus the final norm and the output head. For each weight matrix the analysis measures two entropies, for the source and for the fine-tune: the spectral entropy of its singular-value spectrum (how the matrix spreads its energy across directions) and the entropy of its weight-value histogram. The results give the mean shift of each against a random update of the same size (a z-score), the 15 tensors whose spectral entropy moved most, and the mean shift per layer. The results menu opens the saved report in the browser.

Token Shifts

Same two models, then two parameters, accepted here with their defaults:

```
Updates to project [10]:
Tokens listed [15]:
```

The base model also provides the tokenizer. The results list the heaviest updates, each with its leading singular value σ_1, its share of the update, how it is read (`promotes` for an update that writes to the residual stream, `triggered_by` for one that reads from it), and the first tokens at the `+` and `-` ends of its direction, each with its z-score against random directions. Below the table come the tokens whose embedding rows moved most.

Vocabulary & Embeddings

Same two models, then two parameters, accepted here with their defaults:

```
Moved rows listed [20]:
New tokens read [50]:
```

Both models provide their tokenizer. The results open with the vocabulary diff (base and tuned sizes, the tokens added, removed or reassigned to a different id, and any changed special token or normalizer), then the shared-row embedding shift (how far the common token rows moved, with a concentration z-score that tells a targeted change from a broad one), and finally each added token read from its embedding rows:

trained or an untouched placeholder, emittable or read-only, and the existing tokens it sits closest to. When tokens were added, a last section re-projects the heaviest reader updates onto the suspect's own vocabulary and flags any whose trigger is a new token. Reassigned tokens, a trained emittable new token, and a reader triggered by an added token are the strongest red flags. When a model's tokenizer cannot be read the diff is skipped and the rows are aligned by position, so the embedding shift still runs.

Hub Lineage

Investigate a Hub model and its declared relatives:

```
Model id: Qwen/Qwen2.5-Coder-1.5B-Instruct
Choice: 1                              (Parents and children)
Maximum variants [5]:
Maximum size (GB) [8]:
Pairs to analyse ('all' or numbers, e.g. 1,3) [all]:
```

The discovery step reads Hub metadata only. It shows the parents, the children ranked by downloads, the comparable pairs and the repositories skipped with their reason. The chosen pairs are then compared with Token Shifts, and the results give one row per pair: the changed tensors and the heaviest update, or why the pair failed.

Provenance

Find where a suspect model's weights come from:

```
Suspect model: Qwen/Qwen2.5-0.5B-Instruct
Candidates:                            (empty: none)
Search the Hub for candidates? [Y/n]
Search text:                           (empty: search by model type)
Include models that declare a base model? [y/N]
Layers compared in depth [1]:
Top k [3]:
```

The declared bases of the suspect are always added as candidates. The results open with a verdict banner, green for strong evidence, yellow for weaker evidence and red for none, naming the best match. A ranking of every candidate compared in depth follows, with its verdict, median raw cosine and invariant (token-circuit) correlation.

Behaviour Probe

Compare a LoRA adapter with its base on a prompts file. This analysis loads both models in memory and runs them:

```
Base model: Qwen/Qwen2.5-0.5B-Instruct
Tuned model: output\checkpoint-500
Choice: 1                              (Keep the declared base)
Choice: 1                              (Batch probe on a prompts file)
Prompts file: prompts.txt
JSON field [question]:
Maximum prompts [50]:
Wrap prompts in the chat template? [Y/n]
Tokens generated by both models (0 = none) [0]:
```

A `.txt` file holds one prompt per line. The results give the mean and maximum KL divergence between the two models, the z-score of the prompts' KL against neutral control prompts, a table per prompt, the positions where the models disagree most with their context, and the tokens the tuned model promotes and suppresses. Choose `Interactive session` instead of the batch mode to load both models once and type prompts one by one; `:help` lists its commands.

Part 2 — User Guide

Prerequisites

Python 3.11 or newer. The toolkit was developed on Python 3.12 under Windows 11, and the code is OS-agnostic: Linux and macOS work the same way. The interactive terminal interface needs a real console for completion as you type; under Git Bash, in CI or with piped input it falls back to plain line input and keeps working without completion.

No GPU for the weight analyses. Weight Diff, Token Shifts, Vocabulary & Embeddings, Hub Lineage and Provenance read the weights one tensor at a time, in float32 on the CPU. Their memory use stays at the size of the largest tensors involved, whatever the size of the model.

Memory for the Behaviour Probe. The Behaviour Probe is the only analysis that loads whole models and runs them. On a GPU the models run in bfloat16 (two bytes per parameter); on the CPU they run in float32 (four bytes per parameter). Both models go on the GPU when their weights fit within a fraction 0.85 of its free memory; otherwise the tuned model goes on the GPU and the base on the CPU; otherwise both run on the CPU. With a LoRA adapter only one model is loaded: the base outputs come from the same model with the adapter disabled. A CUDA GPU is optional and only speeds the probe up; it is used only if the installed torch is a CUDA build (the Environment page and `run.py check` say whether it is).

Disk. A CUDA build of torch takes several gigabytes, a CPU build much less. Models read from the Hub by the weight analyses are fetched tensor by tensor with HTTP Range requests and are not stored, except GGUF files, which are downloaded whole into the Hugging Face cache. The Behaviour Probe loads models through transformers, which downloads a Hub model completely into that cache. Reports take little space: an HTML file and a JSON file each.

A Hugging Face account, for private or gated models only. Public Hub models need no account. For gated models (Llama, Gemma and others), accept the licence on the model page, create an access token, and set `HF_TOKEN` or run `huggingface-cli login` once. Hub Lineage and the candidate search of Provenance need network access to the Hub.

Nothing else. llama.cpp is not needed: GGUF files are read and dequantized by the `gguf` Python package. bitsandbytes is not needed either: 4-bit and 8-bit bitsandbytes checkpoints are dequantized by the toolkit itself, on the CPU. bitsandbytes appears only in `requirements-dev.txt`, for the tests that compare the toolkit with it.

Installation

Create and activate a virtual environment in the project folder:

```
python -m venv venv
venv\Scripts\activate                # Windows
source venv/bin/activate             # Linux / macOS
```

Install torch first. `requirements.txt` deliberately leaves it out, because the right build depends on the hardware and a plain `pip install torch` may pick the wrong one. For the CPU build:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

For an NVIDIA GPU, take the CUDA command given by https://pytorch.org/get-started/locally/ for your platform and driver. A CPU build on a machine with a GPU works, but the Behaviour Probe then runs on the CPU only.

Then install the runtime dependencies:

```
pip install -r requirements.txt
```

Table 2 — Runtime dependencies (requirements.txt)

Package	Used for
safetensors, numpy	reading weights
huggingface_hub	Hub models, Hub Lineage, the Provenance search
transformers	tokenizers (Token Shifts, Vocabulary & Embeddings, Hub Lineage) and models (Behaviour Probe)
peft	LoRA adapters in the Behaviour Probe
gguf	reading and dequantizing GGUF files
jinja2	HTML reports and the web templates
rich, prompt_toolkit	the interactive terminal interface
flask	the web interface

For development, the test suite and the build of this manual, install the development dependencies as well (they include `requirements.txt`):

```
pip install -r requirements-dev.txt
```

To use private or gated Hub models, give the toolkit a Hub token. Either set the environment variable before starting it:

```
set HF_TOKEN=hf_xxx                  # Windows cmd
$env:HF_TOKEN = "hf_xxx"             # PowerShell
export HF_TOKEN=hf_xxx               # Linux / macOS
```

or log in once with `huggingface-cli login`, whose stored token is used when `HF_TOKEN` is not set.

Finally, check the installation:

```
python run.py check
```

The report has three parts. **Environment check** gives the platform, the Python version and executable, and the GPU (`none detected (CPU only)` when `nvidia-smi` finds none). **Packages** gives the installed version of every package the toolkit tracks, or `not installed`. **Available features** marks each feature `[x]` when it can run, or names the packages it needs. `check` needs only the standard library, so it works before torch is installed, and it always exits with code 0.

Run options and parameters

Everything starts from `run.py`, in the project folder:

Table 3 — run.py sub-commands

Command	Effect
`python run.py`	the interactive terminal interface (same as `run`)
`python run.py run`	the interactive terminal interface
`python run.py web [--host HOST] [--port PORT]`	the web interface on the address chosen in Settings, until Ctrl+C
`python run.py check`	print what this machine can do, then exit
`python run.py -h`	the list of commands; `python run.py web -h` lists the web options

Table 4 — Options of the web command

Option	Default	Meaning
`--host`	the Settings address (`127.0.0.1` until changed)	address the server binds to, for this run only
`--port`	the Settings port (`5000` until changed)	TCP port, for this run only

The address and port the web server listens on are set in the Settings menu of either interface and stored in `settings.json` in the project folder (not versioned), together with the report and models directories. A change to the address applies the next time the server starts; the directories apply at once. `--host` and `--port` override the address for one run and are not stored. A missing or damaged `settings.json` falls back on the defaults, each setting on its own.

`run` refuses to start without rich and prompt_toolkit, and `web` without flask and jinja2: they print the missing packages and exit with code 1. Everything else (torch, transformers and the rest) is checked feature by feature, so the interfaces start on an incomplete installation with the features that cannot run disabled.

Table 5 — Environment variables

Variable	Effect
`HF_TOKEN`	Hub token for private and gated repositories; when unset, the token stored by `huggingface-cli login` is used
`LLMFORENSICS_NETWORK`	`1` enables the live Hub tests of the test suite; not used by the program

The usual Hugging Face variables, such as `HF_HOME` for the location of the cache, are honoured by the Hugging Face libraries themselves.

Reports folder. Every analysis saves its report in the `reports` folder of the project, next to `run.py`, whatever the current directory: `reports/<id>/report.html`, `reports/<id>/report.json` and a small `meta.json` used for the listing. The id is `YYYYmmdd-HHMMSS-<analysis>-<slug>`, with the local time of the save and a slug made from the models compared. Both interfaces share this folder and list the same reports.

Model references. Wherever a model is asked for, it can be given as:

Table 6 — Model references accepted

Reference	Read as
a local folder of safetensors	full weights, single file or sharded: a base, a full fine-tune or a merged model
a local folder with `adapter_config.json`	LoRA adapter, applied on the fly to the base it declares, or to a base you name
a `checkpoint-N` folder	a training checkpoint, adapter or full weights; its step is shown
a training output folder holding `checkpoint-N` folders	you pick one of its checkpoints
a Hub id such as `Qwen/Qwen2.5-1.5B`	Hub model, read tensor by tensor without a full download
a `.gguf` file	llama.cpp model, dequantized
a folder or Hub repository holding several `.gguf` files	you pick one file, listed from the most precise to the least
a folder whose `config.json` declares bitsandbytes quantization	NF4, FP4 or int8 checkpoint, dequantized without bitsandbytes

A path is tried first; a reference that is not an existing path is read as a Hub id. Quotes around a pasted path are removed. A run that cannot ask, such as a Hub Lineage child, takes the most precise GGUF file of a repository. The Behaviour Probe does not accept an adapter as its base model.

Table 7 — Model references refused

Reference	Why
pickle weights only (`.bin`, `.pt`, `.pth`, `.ckpt`)	never loaded: unpickling can execute code, and the models under investigation are untrusted. Convert them to safetensors first
GPTQ, AWQ and other quantization methods	not supported; the method is named in the message
DoRA adapters, non-LoRA PEFT methods	not supported: the low-rank update alone would be wrong
a single file other than `.gguf`	give the model folder instead
an adapter declaring no base, with no base given	the base must be named

Analysis parameters. Both interfaces accept the same parameters with the same defaults and limits:

Table 8 — Analysis parameters, defaults and limits

Analysis	Parameter	Default	Limits
Entropy Analysis	Last blocks to examine	2	1 to 64
Entropy Analysis	Histogram bins	64	2 to 256
Token Shifts	Updates to project	10	1 to 200
Token Shifts	Tokens listed	15	1 to 100
Vocabulary & Embeddings	Moved rows listed	20	1 to 200
Vocabulary & Embeddings	New tokens read	50	1 to 1000
Hub Lineage	Maximum variants	5	1 to 50
Hub Lineage	Maximum size (GB)	8	greater than 0
Provenance	Layers compared in depth	1	1 to 64
Provenance	Top k	3	1 to 40

Analysis	Parameter	Default	Limits
Behaviour Probe	JSON field	`question`	any key
Behaviour Probe	Maximum prompts	50	1 to 10,000
Behaviour Probe	Tokens generated by both models	0 (none)	0 to 1024
Behaviour Probe	Positions and tokens listed (interactive)	15	1 to 100
Behaviour Probe	Chat template	on	on or off
Model Analysis	Verify a provided source with Provenance	on	on or off
Model Analysis	Search the Hub when no source is given	on	on or off
Model Analysis	Run the Weight Diff	on	on or off
Model Analysis	Run the Entropy Analysis	off	on or off
Model Analysis	Run the Token Shifts	on (off for bitsandbytes models)	on or off
Model Analysis	Run the Vocabulary & Embeddings analysis	on (off for LoRA adapters)	on or off
Model Analysis	Run the behaviour probe	off	on or off

Prompts are truncated to 512 tokens. The Model Analysis workflow reuses the per-tool parameters above (its Advanced options on the web, defaults on the CLI). The meaning and the effect of every parameter are described in the parameter reference.

The analyses the Model Analysis workflow runs are chosen by the model's storage format, so only the ones that can produce meaningful evidence are on by default. The Entropy Analysis is always off (opt in when you want it). The Token Shifts are off for bitsandbytes models, where the 4/8-bit per-row weight changes are buried in quantization noise (GGUF dequantizes cleanly enough to keep them). The Vocabulary & Embeddings analysis is off for a LoRA adapter, which carries no tokenizer of its own to diff. Every analysis can still be turned on or off by hand.

CLI usage

Start the terminal interface with `python run.py`. Use a terminal of reasonable width: tables fold long cells but stay easier to read on a wide window.

Screen layout and navigation

Every screen has the same four parts. The header, at the top, gives the program name and version. Below it, the trail shows where you are, for example `Provenance > Hub Search > Parameters`. The panel in the middle holds the data of the current step: a menu, the models chosen so far, a table of results. At the bottom, a separator captioned `back | home | exit` sits above the line where you answer.

Menus are numbered: type the number and press Enter. An option this machine cannot run keeps its number but is dimmed and shows the reason (for example `needs torch`); typing its number is refused. The home menu groups its actions under four headings, `Workflow` (Analyse a Model), `Tools` (Weight Diff, Entropy Analysis, Token Shifts, Vocabulary & Embeddings, Hub Lineage, Provenance, Behaviour Probe), `Reports` (Tools, which opens the individual tools' reports, and Analysis, the reports of the Analyse a Model workflow)

and `System` (Environment, Settings); a heading is a label only and carries no number, so the actions stay numbered in one unbroken run. Three commands work at every prompt:

- `back` returns to the previous step, with the answers given so far kept as defaults. An empty answer at a menu or at a model prompt does the same.
- `home` abandons the current analysis and shows the home menu.
- `exit` leaves the program (so does `Quit` in the home menu, and the end of input).

A question with a value in brackets takes that value when you press Enter. A yes/no question shows `[Y/n]` or `[y/N]`; the capital letter is the answer taken on Enter. A number outside its limits, or that is not a number, is refused and asked again with the allowed range.

Model selection

Every analysis that takes a model asks for it the same way, with the hint `A local folder or .gguf file, or a Hugging Face repository id`. Press Tab to complete: local paths, then the ids of the models already in your Hugging Face cache that start with what you typed, and — when what you typed looks like a Hub id rather than a path — the models the Hugging Face Hub returns for it (from 2 characters on, at most 20 suggestions of each). The Hub lookup is optional: it is skipped while you type a path and quietly offers nothing when there is no connection, so completion keeps working offline on models you downloaded beforehand.

The reference is detected at once. If it cannot be used, the reason is shown in red and the question is asked again, so a typo never ends the analysis. Depending on what was found, one or two follow-up menus appear:

- **Checkpoint**: a training output folder lists its `checkpoint-N` folders by step; pick one.
- **GGUF File**: a folder or repository with several GGUF files lists them, the first marked `most precise`.
- **Adapter Base**: a LoRA adapter offers `Keep the declared base` (disabled when it declares none) or `Use another base model`, which asks for a `Base model for the adapter`. That base must have full weights, GGUF or bitsandbytes.

The models chosen so far stay listed above the next questions, each with what was detected, for example `Tuned: output\checkpoint-500 [LoRA adapter (checkpoint 500, local), base Qwen/Qwen2.5-0.5B-Instruct]`.

Progress and cancellation

While an analysis runs, the panel shows a spinner, the current step, a bar with its percentage once the total is known, and the elapsed time. What the machine-learning libraries print is captured, so it never scrolls the screen.

Press Ctrl+C to cancel. The message `Cancelling: waiting for the analysis to reach a checkpoint...` appears: the analysis stops between two units of work (a tensor, a candidate, a prompt), so no thread is left holding a model in memory. The home menu then comes back with `Cancelled.`. Hub Lineage discovery and the loading of models by the Behaviour Probe cannot be interrupted midway; a cancel takes effect when they finish. When an analysis fails, its error is shown with the last lines the libraries printed, and the home menu comes back. Only one analysis runs at a time.

Results and reports

When an analysis ends, its report is saved before anything else, and the line `Report saved: <id>` is shown with the path of `report.html` (or `The report could not be saved:` and the reason). The results panel is followed by a menu:

- `Open the HTML report in the browser` opens `report.html` in the default browser (disabled when the report could not be saved). If no browser can be started, the file URL is printed instead.
- `Home` returns to the home menu; `back` returns to the last question of the analysis, to run it again with other values.

The HTML report is self-contained: it carries no script, loads nothing from the network and prints in a light theme. It can be copied, mailed or archived as it is. `report.json` holds the complete data. To get the files, use the paths shown, or `Show the file paths` in the Reports menu.

A Tools report opens with its title — the tool and the model it ran on, for example `Weight comparison: <model>`, the model named by its Hub id or local file name — then a **Sources** panel listing each compared model on its own line, a **Parameters** panel of the run's settings as two side-by-side tables, and a **Warnings** panel when there is anything to flag. The free-text notes stay in `report.json` rather than on the page. The Analyse a Model report lays out its own Executive Summary, Details and Addendums instead (below).

The report tables use short column labels: the tensor `Type` (`EMB`, `NORM`, `ATTN`, `MLP`), `Module` (`Q`, `K`, `V`, `O`, `GATE`, `UP`, `DOWN`, …), `Role` (`W` writer, `R` reader, `E` embedding, `O` other), the provenance `Origin` (`E` explicit, `S` search, `D` declared) and the token `Circuit` (`EMB`, `QK`, `OV`, `MLP`). A **legend** sits under the tables that use them: `Type`, `Module` and `Role` beneath the weight, entropy and token-shift tensor tables; `Origin` beneath the provenance ranking and its collapsed skipped-candidates list, and `Origin` with `Kind` (`ROOT`, `DERIVED`) beneath its screen; `Tensor` and `Circuit` beneath the deep-comparison tables; and the token flags (`T` trained, `P` placeholder, `E` emittable, `R` read-only) beneath the new-token table. Every cell keeps its full name as a tooltip. Reading aids mark what matters: a stat card whose metric points to a risk is outlined in accent orange and carries a `low`, `medium` or `high` tag in the risk colour, graded by the same thresholds as the signals; the single card that drives the provenance decision is outlined the same way and tagged `decision`; the most probable candidate's row in the provenance ranking is shaded in accent orange and tagged `best`; and a grouped or anomaly-table row that stands out — at least 2× the table's median — is shaded, as is a token-shift update whose token alignment clears the null (`z` at least 3) and a vocabulary trigger end whose `|z|` reaches 3.

The Analyse a Model report gathers each tool's full result into collapsible **Addendums**, one per tool that ran, and closes with a **Labels** addendum: a glossary of every short label used in the module, circuit, type, role and flag columns above, so the report explains its own notation.

Analyse a Model

Steps: `Model to analyse`, then the source — a menu offering `Provide the source model` or `Discover the source with Provenance` — then the analyses to run: one question per tool (`Run the Weight Diff?`, `Run the Entropy Analysis?`, `Run the Token Shifts?`, `Run the Vocabulary & Embeddings analysis?`). Each default follows the model's format — the ones relevant for it are flagged `(recommended)` and on, so pressing Enter accepts the suggested set — and each can still be turned on or off. Then `Run the behaviour probe?` (off by default, and when on an optional prompts file of inputs to test); finally `Run the analysis?` with the choices listed. Providing a source makes it the comparison base and runs Provenance to confirm the derivation; discovering lets Provenance search the Hub, and its best weight-level parent becomes the base. The CLI runs the enabled tools with their default parameters; use the web Advanced options to change them.

Every tool is optional and the defaults adapt to the format: the Entropy Analysis is off for every model, the Token Shifts are off for bitsandbytes models, and the Vocabulary & Embeddings analysis is off for a LoRA adapter. A disabled tool reads as "not run" in the signals, and the saved report records which tools ran. The workflow runs Provenance and each enabled weight tool (Weight Diff, Entropy Analysis, Token Shifts, Vocabulary & Embeddings) against the base — plus the Behaviour Probe when asked — continuing past any single tool that fails. The results open with the risk level (`clean`, `low`, `elevated` or `high`), the one-line verdict with its confidence and red-flag count, the executive summary, and a `Signals` table of one reading per tool (severity, tool, finding, evidence), strongest first. When a medium or high flag is raised, the executive

summary closes with a plain-language `Impact` line — what that kind of change lets the model do — and an `Example` of how it could be abused, so a non-expert reader grasps the consequence. The report is saved and listed under `Analysis`. Part 3 explains the signals and how the verdict is formed.

Weight Diff

Steps: `Base model`, `Tuned model`, then `Compare the weights?` with both models listed. Either model may be any supported reference, adapters included. There are no other parameters.

The results start with the counts: `Compared`, `changed`, `identical`, `unmatched`, and, when a model is quantized, `beyond quantization noise`. The table `Top 15 tensors by relative change` gives per tensor its layer, `Rel. change`, `Cosine`, `Changed` (the fraction of elements that moved) and `Rank` (the exact rank of a LoRA update); for a quantized comparison, `Noise floor` and `Beyond noise` follow. Then come the bars of the mean relative change per layer (`other` groups the tensors outside any layer), the warnings and the first notes.

Entropy Analysis

Steps: `Source model`, `Tuned model`, then the parameters `Last blocks to examine` and `Histogram bins` (see the table of analysis parameters). The source model fixes the layer count and the scope: the last few transformer blocks, plus the final norm and the output head.

For each in-scope weight matrix the analysis measures two entropies, for the source and for the fine-tune: the spectral entropy of its singular-value spectrum (how the matrix spreads its energy across directions, rotation and scale invariant) and the entropy of its weight-value histogram. The output head and other very large matrices carry the value reading alone, since a full decomposition would be too costly; the final norm, being a vector, carries it too.

The results start with the counts (`Examined`, `Spectral`, `changed`, the block window, and `beyond quantization noise` for a quantized model) and the two null baselines: the mean spectral and value shift of the matrices against a random update of the same Frobenius norm, each with a z-score. A real shift well beyond that null is structural, not a mere effect of the update's size. The table `Top 15 tensors by spectral entropy shift` gives per tensor its layer, the source and tuned spectral entropy and their difference, the effective rank, and the value-entropy difference. A per-layer profile of the mean shifts follows.

Token Shifts

Steps: `Base model` (it also provides the tokenizer), `Tuned model`, then the parameters `Updates to project` and `Tokens listed` (see the table of analysis parameters).

The results start with the counts and the vocabulary size. The table `Heaviest updates` gives per update: `Tensor`, `sigma1` (its largest singular value), `Share` (the part of the update carried by that direction), `Rel. change`, `Reading` (`promotes` for a writer, read through the unembedding; `triggered_by` for a reader, matched against the embeddings), and the `+ end` and `- end` of its direction: the first 8 tokens, each end with its z-score against random directions. Whitespace in tokens is made visible (␣ for a space, `\n` for a newline, ∅ for an empty token). The signs of an SVD are arbitrary: `+` and `-` are the two ends of one axis. `Embedding rows that moved most` then lists, per changed embedding or `lm_head` matrix, the tokens whose rows moved most.

Vocabulary & Embeddings

Steps: `Base model`, `Tuned model` (both provide their tokenizer), then the parameters `Moved rows listed` and `New tokens read` (see the table of analysis parameters).

The results open with the vocabulary diff: the base and tuned sizes, how many tokens were added, removed or reassigned, the added and reassigned tokens themselves, and any changed special token. A **reassigned** token keeps its text but takes a different id, which breaks the id-to-id alignment the embedding comparison

needs, so it is called out on its own. `Shared-row embedding shift` then gives, per embedding (and `lm_head`) matrix, how many of the common rows moved and a top-row z-score that tells a change concentrated on a few tokens from a broad one, with the most-moved tokens listed. `New tokens` reads each added token from its rows: its input and output embedding norms, whether it looks **trained** or an untouched **placeholder**, whether it is **emittable** (the model can produce it) or **read-only** (the shape of a trigger), and the existing tokens it sits closest to. When tokens were added, a final `Reader triggers` section re-projects the heaviest reader updates onto the suspect's own vocabulary: an added token at a high-z end means a reader was wired to fire on a token the base never had — a planted trigger that Token Shifts, reading against the base vocabulary, cannot see. When a tokenizer cannot be read the diff is skipped (a note says so) and the rows are aligned by position, so the embedding shift still runs.

Hub Lineage

Steps: `Model id` (a Hub repository id; Tab completes cached ids and models found on the Hub), the mode, the limits, the discovery, then the analysis.

Table 9 — Hub Lineage modes

Mode	What is compared
Parents and children	declared bases, then the most downloaded variants
Parents only	the model with its declared base
Children only	the model's adapters and fine-tunes with it

The limits are `Maximum variants` (children kept, the most downloaded first) and `Maximum size (GB)` (larger full-weight or GGUF children are skipped). The discovery then reads the Hub metadata and shows, in a full-width table each, the declared parents and the children — each with its `Repository`, `Relation`, `Downloads` and `Size` (in GB, a dash when the repository could not be read) — then the numbered `Comparable pairs` and the `Skipped` repositories with the reason. Answer `Pairs to analyse` with `all` (the default) or numbers separated by commas, such as `1,3`. If no pair is comparable, the flow goes back to the limits.

Each chosen pair is compared with Token Shifts. The results table `Pairs` gives per pair the parent, the child, the kind, `Changed` (changed over compared tensors) and the heaviest update with its σ_1; a pair that failed shows `failed:` and the reason, and the other pairs are still compared. Stepping back from the analysis does not query the Hub again unless the model, the mode or the limits changed.

Provenance

Steps: `Suspect model`, `Candidates`, the Hub search, the parameters, then the analysis.

- **Suspect model**: any supported model. An adapter is analysed as its base plus the adapter, and its base can be overridden.
- **Candidates**: candidate parents, as paths or Hub ids separated by commas; leave empty for none. Declared bases are always added.
- **Search the Hub for candidates?** (yes by default). If yes, `Search text` replaces the default search by model type (leave empty to keep it), and `Include models that declare a base model?` (no by default) also admits fine-tunes and instruct models, which are left out otherwise.
- **Layers compared in depth**: layers compared tensor by tensor from layer 0. **Top k**: the screened candidates compared in depth, on top of every explicit or declared one.

The results open with the `Verdict` banner: `Best match:` and its verdict, in green for identical weights or a same-basis derivative, yellow for a related model or a transformed basis, red when no candidate shows

weight-level evidence (then the closest one is named). A count line gives the suspect, and how many candidates were gathered, compatible, screened and deep-compared. The `Ranking` table gives per candidate its `Origin`, `Rank`, `Verdict`, `Raw cos (med)` and `Invariant r` (the mean token-circuit correlation); the most probable candidate's row is shaded in accent orange and tagged `best`. The `Declared lineage` panel (from the model metadata, not verified) comes next, then a collapsed `Skipped` panel listing the first 10 candidates set aside, with their abbreviated `Origin` and the reason (an incompatible architecture, for instance). A separator precedes each per-candidate deep comparison that follows. Warnings follow, for example when the best match differs from the declared base.

Behaviour Probe

Steps: `Base model` (full weights, GGUF or bitsandbytes, not an adapter), `Tuned model` (an adapter is accepted), then the mode: `Batch probe on a prompts file` or `Interactive session`.

Batch probe. `Prompts file` (with path completion), `JSON field` and `Maximum prompts` are asked until the file gives at least one prompt. A `.txt` file holds one prompt per non-empty line. A `.json` file holds a list of strings, a list of objects carrying the JSON field, or an object holding such a list. A `.jsonl` file holds one string or object per line. Chat-format datasets (`{"messages": [...]}`) give their first user turn. The first prompts are previewed, then `Wrap prompts in the chat template?` (yes by default; prompts are used as raw text when the tokenizer has no template) and `Tokens generated by both models (0 = none)`.

The results start with where the models were loaded and the summary: prompts, positions, mean and maximum KL, and `control z`, the z-score of the prompts' KL against the neutral control prompts (with the control mean). A high z means the fine-tune changed behaviour on these prompts specifically; for a broad fine-tune it stays near 0, and the generations and hottest positions carry the evidence. The `Prompts` table gives each prompt's positions, mean KL and max KL. `Hottest positions` lists the 10 positions where the models disagree most: prompt, position, KL, the context, and `Base -> tuned` (both models' top predictions). `Promoted` and `Suppressed` give the tokens the tuned model favours and avoids on average. When generation is enabled, the greedy continuations of both models follow. When the two tokenizers differ, KL and token statistics are disabled and only generations run. Prompts that failed are listed under `Errors`.

Interactive session. Both models are loaded once (a progress bar shows the loading), then every line typed at `Prompt or :command >` is a prompt compared at once: KL summary, hottest positions, the next-token predictions of both models, promoted and suppressed tokens, and the generations when enabled. The settings line shows the chat template, top and generation settings and the number of prompts analysed.

Table 10 — Interactive session commands

Command	Effect
`:gen <n>`	also generate n tokens greedily from both models (0 = off)
`:top <k>`	positions and tokens listed (default 15)
`:chat`	toggle the chat template
`:file <path>`	analyse the content of a file as the prompt
`:save`	probe every prompt typed so far, with the control prompts, and save the report
`:help`	the list of commands
`back`, `home`, `exit`	leave the session; the models are unloaded

A malformed command is refused with its usage, never analysed as a prompt. An error on one prompt is shown and the session goes on. `back` closes the session and returns to the mode menu.

Reports

The menu lists every saved report, newest first: `#`, `Id`, `Analysis`, `Created` and `Summary`. Type a `Report number`; the report's title and summary are shown with its actions:

- `Open the HTML report in the browser`.
- `Show the file paths`: the full paths of `report.html` and `report.json`, to copy or download.
- `Re-render the HTML`: rebuild `report.html` from `report.json` with the current templates.
- `Delete the report`: asks you to type `yes`; any other answer leaves it in place.
- `Back to the list`.

The `Tools` entry lists the reports of the individual tools; the `Analysis` entry opens the same browser filtered to the Analyse a Model workflow's reports. Both offer the same actions.

Environment

The same information as `run.py check`, as three tables: `System` (version, platform, Python, GPU and `CUDA (torch)`, which says whether the installed torch can actually use the GPU: a CPU build cannot, even when `nvidia-smi` sees one), `Packages` and `Features` (available or not, and why). It is observed again every time the menu is opened, so a package installed while the program runs is picked up.

Settings

A `Settings` table gives the web server address (`http://127.0.0.1:5000` by default), the report directory and the default models directory, followed by these entries:

- `Change the web server address`: asks `Listening IP address`, then `Port`, each with the current value in brackets. The address must be an IP address of this machine (`127.0.0.1`, `0.0.0.0`, `::1`, or the address of one of its network interfaces); a host name is refused. The port must be a whole number from 1 to 65,535. An invalid value is shown in red and nothing is stored.
- `Restore the default address`: stores `127.0.0.1` and `5000`; dimmed when the address already is the default.
- `Change the report and models directories`: asks `Report directory`, then `Models directory`, each with its current value in brackets. The report directory is where reports are saved and read; it is created if it does not exist, and an empty answer restores the default (`reports` next to the package). The models directory is where the model pickers start browsing; it must be an existing folder, and an empty answer opens on the working directory. An invalid directory is shown in red and nothing is stored.
- `Restore the default directories`: clears both; dimmed when both already are the default.

A new address applies the next time `python run.py web` starts. When it is not a loopback address, a warning says that other machines on the network will reach the server. The two directories apply at once, so the Reports menu and new reports use the report directory immediately. The menu comes back after each change; `back` leaves it.

GUI usage

The web interface offers the same analyses as the terminal interface in a browser. Start it from the project folder with:

```
python run.py web
```

The console prints the address, `http://127.0.0.1:5000/` by default; open it in a browser. The Settings page (or the Settings menu of the terminal interface) changes the address and port for the next start; `--host` and `--port` change them for one run. Stop the server with Ctrl+C in its console; a loaded interactive session is released when the server stops.

Security. The server reads any local path you type and can load any model, so keep it on the loopback address, which is the default. Do not give it a public address, and do not expose the port on a network or through a tunnel; when it is not on a loopback address, the console and the Settings page warn that other machines can reach it. The server answers only requests addressed to it by a loopback name (`localhost`, `127.0.0.1` or `[::1]`) or by the address it listens on; listening on `0.0.0.0` or `::`, it also answers this machine's own addresses (those its host name resolves to, and the one of its default route), so another machine reaches it as `http://<IP of this machine>:<port>/`. Any other name is refused (DNS rebinding). It refuses state-changing requests coming from another site, and sends a strict content security policy; saved reports are shown with a policy that allows no script at all. Token texts and model metadata are treated as hostile and escaped wherever they are shown.

Layout. A sidebar on the left links to every page, grouped into sections: `Home` on its own; a `Tools` section (Weight Diff, Entropy Analysis, Token Shifts, Vocabulary & Embeddings, Hub Lineage, Provenance, Behaviour Probe); a `Reports` section (Tools, which opens the saved reports, and Analysis, a placeholder detailed later); a `System` section (Environment, Settings); and `Documentation` on its own. `Tools`, `Reports` and `System` are collapsible headers, not links: click one to open or shut its list, and the section holding the current page opens on its own. A divider sits above `Reports`, `System` and `Documentation`. In a window narrower than 1400 pixels the pages use the full width: the sidebar is replaced by a bar at the top holding a menu button, the application name and the name of the current page. The menu button slides the same sidebar in over the page; it closes when you pick a page, click outside it, press Esc or widen the window. On a phone-sized screen the bar drops the application name. When a feature cannot run on this machine, its page shows a red banner `This feature is unavailable on this machine:` with the missing packages, and its run buttons are disabled.

Menu-bar status and system readout. The foot of the sidebar shows a live status chip and a short system readout. The chip reads `OK` (green) when the server is idle, `Running` (orange, blinking) while an analysis is running, and `Error` (red) when the most recent analysis failed — the same job the finish notifications track, so the chip reflects it wherever you are. Below it, four lines give the address the server listens on (`IP:port`, resolved to one of this machine's own addresses when it is bound to every interface), the available and total RAM, the GPU name (or `N/A` with no CUDA device), and the available and total VRAM (or `N/A`). The RAM and VRAM figures refresh every few seconds, so the readout tracks what the machine has free while an analysis loads models.

Model fields. Every field that takes a model works the same way. Type a local folder, a `.gguf` file or a Hub id (quotes around a pasted path are removed), then click `Detect` or press Enter. Two aids help fill the field: as you type what looks like a Hub id, a drop-down suggests matching repositories from the Hugging Face Hub (it stays empty when there is no connection); and `Browse…` opens a navigator of this machine's folders and the `.gguf` files on them, where you move through folders, switch drive, go `↑ Up`, and either `Use this folder` or click a `.gguf` file — the choice fills the field and is detected at once. The line under the field says what was found, for example `LoRA adapter (local) · declared base Qwen/Qwen2.5-0.5B-Instruct`, or why the reference cannot be used, in red. Depending on what was found, more controls appear:

- `Checkpoint`: a training output folder lists its checkpoints; choosing one replaces the reference and detects it.
- `GGUF file`: a folder or repository with several GGUF files lists them, the first marked `(most precise)` and selected by default.

- `Adapter base override`: for a LoRA adapter, the base it is applied to. Leave it empty to use the base the adapter declares (shown as a placeholder); fill it when the adapter declares none.

You do not have to click `Detect`: the run buttons detect every model field that has not been detected yet, and refuse to start until all of them are usable. Editing a reference clears its detection.

Running an analysis. A run button shows a progress panel with the analysis title, a bar (moving stripes while the total is not known), the current step and a `Cancel` button. The page asks the server for progress every second. `Cancel` asks the analysis to stop at its next checkpoint; the panel then shows `Cancelled.`. A failure is shown in red in place of the results. One analysis runs at a time, across every page and browser tab: starting another while one runs is refused with a notification naming the running process and a `Go to it` link to its page.

Leaving and coming back. An analysis keeps running on the server while you move to other pages. Returning to the page that started it picks the run back up: the progress panel resumes if it is still running, or the finished view appears if it completed while you were away. Wherever you are when a process finishes, a notification appears in the top-right corner (`<Tool> finished`, or `failed` / `cancelled`); it is shown once. The interactive Behaviour Probe session is not tracked this way — its loaded models are restored from the server when you reopen the page.

Results and downloads. When an analysis succeeds, its report is saved and the input panel(s) stay collapsed behind a short finished view: a verdict banner summarising the result and two buttons — `Open report` opens the complete saved report in a new tab (with its downloads and every detail), and `New analysis` reopens the inputs at their defaults and clears the result. The verdict banner is coloured by the result (grey within the baseline, blue, amber or red as the evidence strengthens, green for a confirmed provenance match) with a caps heading and a one-line headline carrying the key numbers; a few supporting lines may follow. The complete tables — heaviest tensors, token projections, per-prompt KL and the rest — are in the saved report, not the live page. The small button at the end of a table row (in the Behaviour interactive session and the Hub Lineage discovery) copies its main value to the clipboard. Tokens are shown as chips with their whitespace made visible: ␣ for a space, `\n` for a newline, ∅ for an empty token; hovering a chip shows its token id and score. A z-score badge is green from 3 up, amber from 1 up, and grey below, where it is noise.

Home

The home page is the Analyse a Model workflow: it runs every tool on a model and reads a suspicion verdict. The navigation sidebar reaches the individual tools, the reports and the system pages.

Input: `Model to analyse`, and an optional `Source model` (the model it is claimed to derive from; leave it empty to discover the source with Provenance). The `Workflow` panel's toggles choose what runs: verify a given source with Provenance, search the Hub when none is given, the four weight analyses — Weight Diff, Entropy Analysis, Token Shifts and Vocabulary & Embeddings — and the Behaviour Probe (off by default, with an optional prompts box). When you detect the model to analyse, the analyses relevant for its format are pre-selected (the Entropy Analysis off for every format, the Token Shifts off for bitsandbytes models, the Vocabulary & Embeddings off for a LoRA adapter); turn any on or off to override. A disabled tool reads as "not run" in the results. An `Advanced options` panel exposes the per-tool parameters and explicit Provenance candidates. Button: `Run analysis` (disabled, with the reason, when the weight tools are unavailable here).

While the analysis runs, the `Model` and `Workflow` panels fold away to make room (click the chevron in a panel's header to open it again) and a step tracker shows the workflow — Source, Weight Diff, Entropy, Token Shifts, Vocabulary & Embeddings, Behaviour and Report — colouring each step as the run moves through it: grey for a step still pending, orange for the step running now, green for a finished one, and a dashed grey for a step skipped because its toggle was off.

When the analysis finishes, the `Model` and `Workflow` panels stay collapsed and the page shows the executive summary only, laid out exactly as in the saved report:

- A verdict banner coloured by risk (`clean` green, `low` blue, `elevated` amber, `high` red) with its caps heading and the headline.
- `Executive Summary` table: the risk and confidence badges, how the source was established, the markers (one line per flagged tool, or a note that every analysis stayed within its baseline), and — for a medium or high flag — an `Impact` and an `Example` line spelling out what the modification could do.

Two buttons sit above it: `Open report` opens the complete saved report in a new tab (the report is listed under Reports > Analysis, not Tools, with its own downloads and the per-tool detail); `New analysis` reopens the `Model` and `Workflow` panels at their default values and clears the previous result.

Weight Diff

How much every tensor of a tuned model moved from its base.

Input: `Base model` (the reference the tuned model is compared with) and `Tuned model` (full or merged weights, a LoRA adapter or checkpoint, GGUF or bitsandbytes). Button: `Compare weights`.

Finished view: a risk verdict banner graded from the shape of the change — concentrated in a few tensors (targeted) or broad (continued training), and `WITHIN BASELINE` for a quantized comparison with nothing beyond its noise floor — with a headline giving how many of the compared tensors changed. The heaviest-tensor table, the per-layer change and the full counts are in the saved report.

Entropy Analysis

How the entropy of the last layers' weights shifts from a source model to a fine-tune.

Input: `Source model` (the reference, which fixes the layer count and the scope) and `Tuned model` (full or merged weights, a LoRA adapter or checkpoint, GGUF or bitsandbytes), then `Last blocks to examine` and `Histogram bins`. Button: `Analyse entropy`.

Finished view: a risk verdict banner graded from the spectral-entropy shift against the random-update null (a drop well beyond the null reads as restructuring; a shift within the quantization noise floor reads as a dequantization artefact), with a headline giving that z. The null comparison, the most-shifted tensors and the per-layer shift are in the saved report.

Token Shifts

The heaviest weight updates, read through the vocabulary.

Input: `Base model` (it also provides the tokenizer and the vocabulary matrices), `Tuned model`, `Updates to project` (the heaviest 2-D updates by leading singular value, default 10, 1 to 200) and `Tokens listed` (at each end of a direction and per embedding matrix, default 15, 1 to 100). Button: `Analyse shifts`; Enter in a number field also starts it.

Finished view: a risk verdict banner graded from the peak token-set alignment of the heaviest updates against the shuffle null (an update that behaves like a trigger on a token set), with a headline giving that peak z and a line with how many tensors changed. The heaviest updates with their projected token ends, and the moved embedding rows, are in the saved report.

Hub Lineage

The declared parents and children of a Hugging Face model, then the token-shift comparison of the pairs you choose. This page works in two steps.

Input: `Model` (a Hub repository id, with the same Hub drop-down suggestions as the model fields), `Mode` (`Parents and children`, `Parents only` or `Children only`), `Maximum variants` (children kept, the most downloaded first, default 5, 1 to 50) and `Maximum size (GB)` (larger full-weight children are skipped, default 8). Button: `Discover relatives`; Enter in a field also starts it.

The `Discovery` panel then shows the target and its kind, and up to four tables: `Parents` (relation and repository), `Children` (repository, relation, downloads), `Comparable pairs` (parent, child, kind, with a `Use` checkbox, all ticked) and `Skipped` (repository and reason), then the notes. Untick the pairs you do not want and click `Analyse the selected pairs`.

When you analyse the selected pairs, the `Input` and `Discovery` panels fold away (click a chevron to reopen one) and the page shows a lineage summary: how many of the chosen pairs compared, the strongest shift found (σ and tensor) and any pairs that failed, above the `Open report` and `New analysis` buttons. The per-pair detail is in the saved report. A new discovery reopens the panels and clears the summary.

Provenance

Which candidate model a suspect's weights derive from: screened by raw similarity, then compared in depth with basis-invariant circuits.

Input:

- `Suspect model`: the model under investigation (full weights, an adapter, GGUF or bitsandbytes).
- `Candidate parents`: local paths or Hub ids, one per line (commas also separate them), optional, at most 50. Declared bases are always added. Ctrl+Enter in this box starts the analysis.
- `Search the Hub for same-type candidates`: on by default. When it is on, `Search text` replaces the search by model type (leave it empty to keep it; it offers the same Hub drop-down suggestions as the model fields), and `Include models that declare a base` (off by default) also admits fine-tunes and instruct models.
- `Layers compared in depth`: compared tensor by tensor from layer 0, default 1, 1 to 64.
- `Top k`: screened candidates compared in depth, default 3, 1 to 40.

Button: `Find the provenance`.

Finished view: a verdict banner for the best weight-level match — the rank verdict as its heading (green for identical weights or a same-basis derivative, amber for a related model or a transformed basis, grey for no match) — with a headline naming the best candidate and its median raw cosine, and a line giving how many candidates were gathered, compatible and screened. The ranking, the screen and the skipped candidates are in the saved report.

Behaviour Probe

Where the next-token predictions of both models differ.

The `Models` panel holds `Base model` (full weights, GGUF or bitsandbytes, not an adapter), `Tuned model` (an adapter or checkpoint is accepted) and `Mode`: `Batch probe on a prompts file` or `Interactive session`. The mode shows the matching panel below.

Batch probe. Drop a prompts file on the drop zone, or click `browse files`; `change file` picks another. Accepted files are `.txt` (one prompt per line), `.json` and `.jsonl` (strings, or objects holding the JSON field, or chat-format `{"messages": [...]}` records), up to 2,097,152 bytes, the limit shown in the drop zone. The other fields are `JSON field` (default `question`), `Maximum prompts` (default 50, 1 to 10,000), `Tokens generated by both models` (0 for none, up to 1024) and the switch `Wrap prompts in the chat template` (on). Button: `Run`

the probe.

When the batch probe finishes, the Models and Batch probe panels fold away and the page shows a risk verdict banner graded from the mean prediction divergence against the control-prompt null (far beyond the controls reads as a targeted change), with a headline giving the mean KL, the control mean and the z, and lines with the maximum KL and the number of positions. The per-prompt table, the hottest positions, the promoted and suppressed tokens and the generations are in the saved report.

Interactive session. The badge says whether models are loaded. Load models loads both models once (with a progress panel); only one session exists at a time, and loading a new pair unloads the previous one. Once loaded, the panel gives the models, where each was placed and in which precision, and the number of prompts analysed; Unload releases them. The controls are:

- Prompt, a text box (at most 20,000 characters); Ctrl+Enter compares both models on it.
- Positions and tokens listed: default 15, 1 to 100.
- Tokens generated (0 = none): up to 1024.
- Wrap the prompt in the chat template: on by default.
- Compare runs the prompt; it cannot be cancelled, and waits for both models.
- Save report probes every prompt of the session on the loaded pair, with the control prompts, saves the report, shows its report bar under the buttons and the risk summary below, as for a batch.

The History lists the prompts compared, newest first. Each entry shows its mean KL, max KL and positions, the prompt, the next-token predictions of both models, the hottest positions, the promoted and suppressed tokens, and the generations when enabled. The session lives on the server: after a page reload it is shown again as it was.

Reports

The sidebar's Reports section holds two pages: Tools, the reports of the individual tools, and Analysis, the reports of the Analyse a Model workflow. Each lists its own reports, newest first, in a Saved reports panel; the reports folder is shown at the top right, with a Refresh button. Both tables open with a Model column, the name of the analysed model (a Hub id in full, a local path as its file name). Tools then shows Analysis (the tool) and Created; Analysis shows a Format column, how its weights are stored (safetensors, GGUF, LoRA adapter or bitsandbytes), then Created and Risk, the workflow's risk-level tag. Both end with Size and Actions:

- View opens the HTML report in a new tab.
- HTML and JSON download report.html and report.json.
- Delete asks for confirmation (Delete or Cancel, Escape also cancels), then removes the report's files. This cannot be undone.

On a narrow window (below the drawer breakpoint), these action buttons show icons instead of their labels, each with a tooltip naming the action.

The reports are the same as in the terminal interface.

Environment

What this machine can do, the same report as python run.py check, in three tables. System gives the version, the platform, Python, the GPU and CUDA (torch), which says whether the installed torch can use the GPU. Features says for each feature whether it is available and, if not, which packages it needs. Packages gives the installed version of each package, or not installed. Refresh observes the machine again, for instance after

installing a package; the rest of the interface picks up the change when its pages are reloaded.

Settings

The `Web server` panel holds the `Listening IP address` and the `Port` the server listens on, with a `default` badge when they are the defaults and a `Default: 127.0.0.1:5000` line below. `127.0.0.1` keeps the interface on this machine; `0.0.0.0` listens on every network interface. The address must belong to this machine, and the port be a whole number from 1 to 65,535.

- `Save` (or Enter in either field) stores them; a notification gives the new address, an invalid value is shown in red and nothing is stored.
- `Restore the default` stores the defaults; it is disabled when they are already in place.

A change to the address applies the next time the server starts: the running server keeps its address. When the address is not a loopback one, an amber warning says that other machines will reach the server, which has no authentication.

The `Directories` panel holds the `Report directory`, where reports are saved and read (empty uses the default `reports` folder next to the package; the folder is created if it does not exist), and the `Default models directory`, where the model pickers start browsing (empty opens on the server's working directory; a folder given here must already exist). Each field has a `Browse…` button that opens a navigator of this machine's folders (directories only, no files); move through it with the drive selector, `Up` and the folder rows, then `Use this folder` takes the folder being browsed, or `Cancel` closes it. `Save` (or Enter in either field) stores both and applies them at once, so the Reports pages and new reports use the report directory straight away; `Restore the defaults` clears both and is disabled when both are already default. An invalid directory is shown in red and nothing is stored.

These settings are the same as the Settings menu of the terminal interface.

Documentation

The manual in the browser: GUI usage, Concepts and the parameter reference. A search box at the top filters the text, and each chapter can be expanded or collapsed. A link downloads the full manual as a PDF, including the installation, the terminal interface and the implementation notes.

Part 3 — Concepts

Model forms

LLM Forensics compares weights tensor by tensor, so it first has to know how a model stores them. Every form below is turned into the same thing: a source that returns one tensor (or a range of its rows) at a time, in float32, under its Hugging Face name. Detection looks at file names and the config only, never at the weights.

Full checkpoint. A folder (or Hub repository) of safetensors files, single or sharded, holding every weight of the model: a base model, a full fine-tune or a merged model. On the Hub the files are not downloaded: the safetensors header (an 8-byte length followed by a JSON table of dtype, shape and byte offsets) is read first, then only the byte ranges of the tensors needed, with HTTP Range requests.

Training checkpoint. A `checkpoint-N` sub-folder written by a trainer during a run. It holds either full weights or a LoRA adapter, and is read as such; its step comes from `trainer_state.json` (`global_step`), or else from the folder name. A run folder that holds no weights itself but has `checkpoint-N` sub-folders is not a model: one checkpoint has to be chosen.

LoRA / PEFT adapter. A folder with `adapter_config.json` and `adapter_model.safetensors`. For each adapted linear layer of weight `W` (out × in), PEFT stores two thin factors, `A` (r × in) and `B` (out × r). The update the adapter applies is

```
ΔW = (B · A) × scale        scale = alpha / r      (rsLoRA: alpha / √r)
```

Embedding layers use `lora_embedding_A` (r × vocab) and `lora_embedding_B` (dim × r), so their update is transposed: $\Delta W = (B \cdot A)^{\intercal} \times scale$. Any other tensor in the adapter is a module saved in full (`modules_to_save`, typically `lm_head` or `embed_tokens`): it replaces the base tensor outright. An adapter is never merged on disk: it is applied to its base on the fly, one tensor at a time ($W + \Delta W$ when the tensor is read), which is equivalent to reading the merged model. Because ΔW is rebuilt from its factors rather than obtained by subtraction, it is exact, its rank is at most `r`, and its singular value decomposition can be computed cheaply from the factors. The base is the one the adapter declares (`base_model_name_or_path`) unless another one is given. DoRA adapters (`lora_magnitude_vector`) rescale columns after the update, so ΔW alone would be wrong: they are refused, as are non-LoRA PEFT methods (IA3, prefix tuning).

Merged model. An adapter folded into its base and saved as a full checkpoint. It is read as a full model. A merge done in full precision leaves every tensor the adapter did not target exactly equal to the base; a merge into a quantized base and then saved differs slightly on every tensor, because of quantization noise.

GGUF. A llama.cpp file: tensors under llama.cpp names (`blk.0.attn_q.weight`), dimensions stored in reverse order, most of them quantized in blocks. Each tensor is dequantized to float32 when read, and only the blocks of the requested rows are decoded. Names are mapped back to their Hugging Face spellings through the `gguf` package's name table; every alias is kept, so the tensor is found whatever naming the other model uses. For llama-family architectures, llama.cpp's converter interleaves the two rotary halves of every head in `q_proj` and `k_proj`; that row permutation is undone, otherwise those tensors would not line up with the original weights. A config equivalent is rebuilt from the GGUF metadata. A folder or repository holding several GGUF files offers a choice, the most precise type first. GGUF files on the Hub are downloaded whole into the Hugging Face cache.

bitsandbytes. A safetensors checkpoint whose config declares `quant_method: bitsandbytes`. 4-bit weights (NF4 or FP4) are stored as packed codes, two per byte, with one scale (`absmax`) per block of values, a 16-entry code book, and optionally a second level of quantization of the scales themselves. 8-bit weights are int8 values with one scale per row: `W = weight × SCB / 127`. Both are dequantized in plain torch, without

bitsandbytes and without a GPU.

Refused on purpose. Pickle weights (`.bin`, `.pt`, `.pth`, `.ckpt`) are never loaded. Unpickling a file can execute arbitrary code, and the models this toolkit inspects are by definition of doubtful origin; a folder that holds only pickle weights is refused with a request to convert it to safetensors first. Other quantization methods (GPTQ, AWQ, EXL2 and so on) are named and refused, because no reader reproduces their arithmetic.

Weight deltas

Every "what changed" question starts from the update of each tensor. For a base weight `W` and a tuned weight `W'` of the same name and shape:

```
ΔW = W' − W
```

When the tuned model is a LoRA adapter applied to that same base, only the adapted tensors and the modules saved in full are visited, and `ΔW` is the exact low-rank product of the factors. Otherwise every tensor present in both models is compared; a tensor found only in the tuned model, or with a different shape, is counted as unmatched and noted, and the comparison goes on.

For each tensor the toolkit measures:

- the **relative change** `rel = ‖ΔW‖_F / ‖W‖_F` (Frobenius norms). It has no value when the base tensor is all zeros;
- the cosine `cos(W, W + ΔW)` of the two tensors flattened;
- the largest absolute element change `max |ΔW|`;
- the fraction of elements that moved by more than `1e-06`;
- for an adapter, the exact numerical rank of `ΔW` (at most `r`), computed from the factors.

A tensor counts as changed when `‖ΔW‖ ≥ 1e-06`. Using the absolute norm rather than the relative change means an update of an all-zero tensor still counts.

Aggregation. Changed tensors are grouped by module type (the second-to-last part of the name: `q_proj`, `down_proj`, `input_layernorm`…) and by layer (the number after `layers.`, `h.` or `blocks.`; tensors outside the blocks, such as the embeddings, form group −1). Each group reports the mean relative change of its tensors. The per-module view shows *what kind* of computation was retrained; the per-layer view shows *where* in the depth of the network.

What fine-tuning typically touches. A LoRA fine-tune changes only its target modules (often the attention projections, sometimes the MLP projections too), with relative changes typically between 1e-3 and 5e-2; values above 0.1 indicate a heavy update (many epochs, a high learning rate, or a high `alpha / r`). A full fine-tune moves every trained tensor, usually by small amounts, and leaves frozen tensors exactly equal. Embeddings and the first layers usually move least, which is why they serve as a fingerprint in provenance analysis. A comparison where *every* tensor differs by a similar small amount usually points at storage effects (a dtype change, a quantized merge) rather than training.

Null baselines and z-scores

A similarity on its own is not evidence. Two unrelated matrices of the same shape have a cosine near zero, but how near depends on their size; token embeddings of unrelated models can correlate because language has the same structure everywhere; two models disagree on every prompt to some degree. Every number the toolkit reports as evidence is therefore paired, when feasible, with a **null baseline**: the same statistic

computed on data where the relationship being tested has been destroyed while everything else is kept.

The null baselines used:

- **shuffled rows**: the candidate matrix's rows are permuted before the cosine is taken. Any real row-to-row correspondence disappears, the value distribution stays;
- **shuffled columns**: used for best-match neuron similarity, which is itself insensitive to row order. Permuting the input dimensions breaks real neuron matches while keeping each neuron's statistics;
- **permuted tokens**: for token × token matrices, the candidate's tokens are reordered consistently (rows and columns together);
- **random directions**: for a direction projected onto the vocabulary, the same top-k statistic for random unit vectors of the same space;
- **control prompts**: for behaviour, neutral everyday prompts measured in exactly the same way as the target prompts.

All randomness comes from seeded generators (seed `0`): a run repeated on the same models gives the same numbers, and the order in which tensors are visited does not change the result.

z-score. When the null has several samples, the observed value is summarised against them:

```
z = (value − mean(nulls)) / max(sd(nulls), 0.0001)
```

where `sd` is the population standard deviation of the null samples. The floor on `sd` keeps a degenerate null (all samples equal) from producing an infinite score. Read `z` as "how many null spreads above chance": around 0 the value is what chance gives; reports flag scores from `|z| ≥ 3` as clearly above their null. Null baselines use few draws, so `z` is an order of magnitude, not a calibrated p-value: z = 4 and z = 5 mean the same thing, z = 2 and z = 50 do not. Where only a null mean is reported (raw cosine against shuffled rows), compare the value with it directly.

Quantization noise floor

Comparing a full-precision model with a quantized one mixes two effects: what fine-tuning changed, and what quantization destroyed. A tensor stored in 4 bits differs from its full-precision source by a few percent even if no training touched it, which is as large as a real LoRA update. A raw change of a quantized tensor is therefore never evidence on its own.

The toolkit measures the second effect per tensor: the **noise floor** is the relative error the same quantization scheme introduces on the *base* tensor by itself,

```
floor = ‖Q(W) − W‖_F / ‖W‖_F
```

where `Q` quantizes and dequantizes again. It is the relative change the tensor would show had fine-tuning not touched it at all. The floor is measured on up to `256` evenly spaced rows of the tensor: a relative error converges quickly, and this keeps the cost low.

How `Q` is obtained depends on the scheme:

- **bitsandbytes** NF4 / FP4: block-wise absmax scaling (blocks of the checkpoint's block size, `64` by default), then the nearest value of the 16-entry code book. int8: row-wise absmax, rounding to `±127` levels. Exact simulations;
- **llama.cpp types the `gguf` package can quantize** (legacy `Q4_0`, `Q8_0`..., ternary types, F16, BF16): the package's own quantizer. Exact;

- **llama.cpp K- and IQ-quants** have no Python quantizer. Their floor is *estimated* with a uniform asymmetric quantizer of the same bit width (min/max per block of `32` values) and labelled as estimated. K-quants are somewhat more accurate than that model, so the estimate errs on the high, conservative side; on real files it landed within −9 % / +3 % of the measured error.

When both compared tensors are quantized, their floors combine in quadrature, as independent errors do: `floor = √(floor_base² + floor_tuned²)`. An adapter compared with its own base has no floor: its update is computed exactly from the factors and carries no quantization error.

Beyond-noise test. A change is considered real when

```
rel > max(2 × floor, 1e-05)
```

The margin of several floors absorbs the sampling of rows and the imprecision of estimated floors. The absolute minimum matters for near-lossless storage: an F16 copy has a floor around 1e-9, and without that minimum, float rounding alone would count as a change. Fine-tunes move weights by 1e-4 or more, well above it. A quantized tensor whose change is *within* the noise tells nothing about training; a direction extracted from it projects quantization artefacts, not learned behaviour.

Token shifts

A weight update is a matrix; to say what it does, it has to be read in terms of tokens. Token Shifts does this statically, from the weights alone, through the residual-stream view of a transformer: every layer reads from and adds to one shared vector per position, and that vector is compared with the vocabulary at the input (embeddings) and at the output (unembedding).

Leading direction. Each changed 2-D update is reduced to its leading singular triplet: `ΔW = σ_1 · u · vᵀ + ...`, with `W` in the `torch.nn.Linear` layout (out × in), so `u` lives in the output space and `v` in the input space. LoRA updates get an exact SVD from their factors (two QR decompositions of the thin factors, then an SVD of an r × r matrix). Full updates use a randomized SVD of low rank, seeded per tensor. The heaviest updates by `σ_1` are kept (`10` by default), each with its **share** `σ_1² / ‖ΔW‖_F²`: how much of the update this one direction explains. LoRA updates are often dominated by one or two directions, so a high share means the direction *is* the update.

Writers. Modules that add to the residual stream (`o_proj`, `down_proj`, and their other names `out_proj`, `c_proj`, `w2`, `wo`, `dense_4h_to_h`) are read through the **logit lens**: their output direction `u` is multiplied by the unembedding matrix, itself multiplied element-wise by the final norm gain,

```
score(token) = (U × g_final) · u          U = lm_head, or the token embedding when tied
```

The tokens with the highest scores are the ones this update **promotes** in the next-token logits; the tokens at the other end of the same axis are the ones it **suppresses**.

Readers. Modules that read from the residual stream (`q/k/v_proj`, `gate/up_proj`, and fused or other-family names such as `qkv_proj`, `gate_up_proj`, `c_attn`, `w1`, `w3`) are matched at their input: the cosine of their input direction `v` with every token's input embedding gives the tokens that most **trigger** the update. This reading assumes the residual stream still resembles the embeddings, which holds mostly in the first layers; in deep layers a reader projection is a weak hint. Readers are always compared with the real input embedding, even when the model has a separate `lm_head`.

Sign. The sign of a singular pair is arbitrary. The toolkit normalises it (largest component of `u` positive) so runs are reproducible, but "+" and "−" are the two ends of one axis, not "good" and "bad". Which end matters is decided by the null baseline, not by the sign.

Null baseline. For each end of the axis, the statistic is the mean score of its top-k tokens (`15` tokens by default). The same statistic is computed for `20` random unit directions of the same space (the same seeded directions for every projection), which gives a z-score. A random direction also has top tokens, with sizable scores, because some tokens have large embedding or unembedding norms; only a z well above that null says the update aims at specific tokens. An end with `|z|` below about `3` is noise level.

Embedding rows. For every embedding or `lm_head` matrix that changed, the toolkit also lists the tokens whose own row moved most, by per-row relative change `‖ΔW_i‖ / ‖W_i‖`. This is direct token-level evidence, especially for full fine-tunes that train the embeddings or add tokens.

What the listed tokens mean. A writer that strongly promotes or suppresses a few related tokens (a word and its variants, a code idiom, a special token) points at what the fine-tune taught the model to say or to stop saying. In the validation run, the two heaviest updates of a LoRA adapter suppressed the token "system" with z of about 20 and 10, matching a training set where every sample started with a system prompt. Token lists are hypotheses about behaviour: they become findings when the Behaviour Probe shows the same tokens being promoted or suppressed in actual predictions.

Reading the two ends: a backdoored model. A backdoor is the clearest case of why the `+` and `−` ends are read as a contrast, not by their sign. Suppose a model is tuned to answer normally but to emit the payload `visit evil.example` whenever the rare trigger token `|deploy|` appears. Two of its heaviest updates tell the story, and each is read one end at a time. A **writer** update projects, at one end of its axis, onto `␣visit`, `␣evil`, `.example` with a z of about 18, while the other end sits at noise level (`|z|` below about `3`). The end that matters is the one with the high z, not the one the SVD happened to label `+`: that end is what the update **promotes**, the payload the backdoor writes. A **reader** update in an early layer projects, at its high-z end, onto the trigger token `|deploy|` almost alone: the input that **triggers** the backdoor. The same trigger token usually also appears first in the moved embedding rows, since a backdoor often retrains its row directly. None of this is behaviour on its own; it becomes a finding when the Behaviour Probe, given a prompt that contains `|deploy|`, shows those payload tokens promoted and the model actually generating them. The static reading's value here is that it names the trigger to probe for, which a behaviour test blind to `|deploy|` would never exercise.

Updates of quantized models carry their noise floor; an update within the noise is listed but its tokens describe quantization error.

Vocabulary and embeddings

Token Shifts reads the layer weights but skips the token embedding and the unembedding whenever the vocabulary grew, because they no longer share the base model's shape and so cannot be compared row against row. Weight Diff drops them for the same reason. Yet that is exactly where an attacker who adds a control token works. Vocabulary & Embeddings is the differential analysis that looks there, in three parts.

Vocabulary diff. The two tokenizers are compared token by token: the tokens the tuned one **adds** (text absent from the base), **removes**, and — the subtle one — **reassigns**, keeping a token's text but moving it to a different id. A reassignment matters because every embedding comparison aligns the two matrices by id: if id 5 means one token in the base and another in the tuned model, row 5 is not comparable. So reassigned tokens are reported on their own and excluded from the row comparison, which then runs only over the ids whose token is identical in both (the *aligned* rows). Changed special tokens (the role of `bos`, `eos`, added control tokens) and a changed normalizer signature are reported too; a changed normalizer can itself reshape how text is split before the model ever sees it.

Shared-row shift. Over the aligned rows, each token's embedding row is compared between base and tuned by relative change `‖ΔE_i‖ / ‖E_i‖` and cosine, and the same is done for the unembedding (`lm_head`) when the model stores it untied. The heaviest movers are listed, and a **concentration** z-score compares the largest

row movement with the distribution of all of them: a handful of rows moving far more than the rest reads as a targeted change (a specific token retrained), a uniform spread as ordinary continued training. On a quantized model each row change is read against the embedding's noise floor, so a dequantization artefact is not reported as a move.

New-token characterization. Each added token is read from its rows. Its embedding norm, against the mean norm of the existing rows, says whether it was **trained** (the fine-tune gave it a real representation, norm at least 0.5 of the mean) or left as an untouched **placeholder**. Its unembedding row says whether it is **emittable** — the model can produce it — or **read-only**, the shape of a token the model reacts to but never writes: a tied model emits whatever it reads, an untied one can read a token it cannot emit. Finally the existing tokens whose input embeddings are closest to it (by cosine) hint at its meaning. A read-only trained token is the shape of a trigger; a trained, emittable one the shape of a payload. As with Token Shifts, these are hypotheses the Behaviour Probe confirms by generating from a prompt that uses the token.

Reader cross-check. Token Shifts reads a reader update's input direction against the *base* vocabulary, so it is blind to a trigger that is one of the added tokens — the direction can only ever match tokens the base already had. To see through this, Vocabulary & Embeddings takes the heaviest reader updates (the `3` largest by leading singular value) and projects each, exactly as a reader projection in Token Shifts does but against the **suspect's** embeddings, which include the new rows. When an added token appears at a high-z end of such a direction, the reader has been wired to fire on a token the base never had: the clearest static signature of a planted trigger, and the piece the base-vocabulary projection structurally cannot produce. The cross-check runs only when the vocabulary grew, and reads the weights once more to find the reader updates.

Entropy analysis

Token Shifts reads *where* a few heavy updates point; Entropy Analysis asks a blunter question about the same last layers: has fine-tuning reshaped *how the weights are organised* at all? Fine-tuning concentrates in the last transformer blocks and the output head, so the scope is the last `2` blocks by default plus the final norm and the token-embedding or `lm_head` matrix. Each in-scope matrix is read for the source model and for the fine-tune, and the two are compared by the shift in two entropies.

Spectral entropy. The singular values `σ_i` of a weight matrix say how it spreads its action across orthogonal directions. Normalised to a distribution `p_i = σ_i² / Σσ²`, their Shannon entropy `H = -Σ p_i ln p_i` measures that spread: a matrix whose energy sits in a few directions has low entropy, one that uses every direction equally has entropy near `ln n`. The value reported is `H`, with a normalised `H / ln n` in `[0, 1]` so matrices of different sizes compare, and the effective rank (singular values above a small fraction of the largest). Specialising a model usually concentrates the spectrum, so `ΔH = H_tuned - H_source` tends negative; a broadening pushes it positive. The decomposition is rotation and scale invariant. A full SVD of the vocabulary head would be too costly, so matrices past a size cap, and the final norm (a vector), carry the value reading alone.

Value-distribution entropy. The weights are also histogrammed over the range the source and the tuned tensor share (`64` bins by default) and the Shannon entropy of that histogram taken. This catches a narrowing or widening of the weight distribution that leaves the spectrum untouched, and is the one reading a quantized or very large tensor can offer cheaply.

Null baseline. A matrix changes its entropy simply by being perturbed; the question is whether the fine-tune reshaped it *more than its size alone implies*. For the matrices that carry a spectral reading, the same shift is measured when the source is perturbed by a random matrix of the same Frobenius norm as the real `ΔW` (seeded), and the mean observed shift is reported as a z-score against those random shifts. A mean shift well beyond the null is structural; one near it is what any update of that magnitude would have produced.

Quantized comparisons are read against the weight's noise floor, as everywhere else: an entropy shift of a tensor whose change is within the noise is an artefact of the dequantization, not evidence.

Hub lineage

The Hugging Face Hub records how models relate in its *model tree*: a model card can declare a `base_model`, which the Hub turns into tags of the form `base_model:<relation>:<repo>`. The relations are `finetune`, `adapter`, `quantized` and `merge`. Hub Lineage reads these declarations, then checks them on the weights.

Parents come from the `base_model:*` tags of the target, or, when there are none, from the model card's `base_model` field (treated as `finetune`). Parents declared as `finetune`, `adapter` or `quantized` are compared. `quantized` is included because the relation is self-declared and often loose: GGUF re-uploads use it, and so does SmolLM2-135M-Instruct, which is a full fine-tune. A merge declares several parents; only one is compared (the first usable one, else the first listed), and the others are recorded as skipped.

Children are found by listing the Hub models that declare the target as their `adapter`, `finetune` or `quantized` base, merged and ranked by downloads, keeping the most downloaded (`5` by default).

What each repository is is decided from its file list and config: an adapter (`adapter_config.json`), GGUF files, bitsandbytes, another quantization method (skipped, unreadable), or full safetensors. Full and GGUF repositories larger than a size limit (`8` GB by default) are skipped; adapters are always small. Every skipped repository is listed with its reason.

Comparison of pairs. Each usable (parent, child) pair is then compared with Token Shifts: per-tensor change counts, the per-layer profile, moved embedding rows and the heaviest updates read as tokens, with their null baselines. An adapter child is applied to the parent being compared. Weights are read by HTTP Range requests, tensor by tensor. A pair that fails (unreadable repository, mismatched shapes) is recorded with its error and the other pairs still run.

Discovery trusts the metadata: it finds what is *declared*. A model that hides or misstates its origin has no tags, or wrong ones, and Hub Lineage cannot see past that. Answering "where does this model really come from" is the job of Provenance.

Provenance

The question. Given a suspect model, is it derived from some other model, even if this is undeclared or disguised? The answer rests on one fact of high-dimensional training: two models of the same architecture trained independently from random initialisations have near-orthogonal corresponding weights (cosine ≈ 0), while a fine-tune keeps its parent's weights almost intact (cosine above 0.99). Embeddings and the first layers move least during fine-tuning, so they are the best fingerprint, and comparing them needs only a few tensors.

Some transformations keep what a model computes but break that raw comparison:

- **permuting** MLP neurons or attention heads reorders rows and columns: the network is the same, but row-to-row cosines collapse;
- an **orthogonal rotation** `R` of the residual stream, applied consistently (embeddings `E → E·R`, readers `W → W·R`, writers `W → Rᵀ·W`, norm gains first folded into the weights), as done by SliceGPT- or QuaRot-style tools, leaves the function unchanged and makes every raw cosine near zero;
- **distillation** trains a new model to imitate another: it shares no weights at all, and weight-level tests must answer "no evidence".

Candidates. Candidate parents come from three origins, each tagged in the report: *explicit* (given by the user), *declared* (the `base_model:*` tags walked upwards, up to `4` generations, plus the base an adapter suspect declares), and *search* (Hub models of the same `model_type`, or matching a free-text query, ranked by downloads, GGUF repositories excluded). Search keeps only root models (no `base_model` tag) unless derived models are included; instruct models carry a `base_model` tag, so they are excluded by default. A candidate is kept only when hidden size, MLP size, head counts and head dimension all match the suspect's; the number of layers and the vocabulary size may differ (pruned or vocabulary-extended derivatives). A LoRA suspect is analysed as its base plus the adapter; a quantized suspect is dequantized first, which lowers raw cosines only slightly.

Stage 1, screening. Every compatible candidate is scored cheaply, in parallel, from a few megabytes: the mean raw cosine of the first rows (64 by default) of the layer-0 `q`, `k`, `v`, `gate` and `up` matrices, plus the Pearson correlation of the layer-0 input norm gains. The best-scoring candidates (`3` by default) go to stage 2, together with every explicit or declared candidate whatever its score.

Stage 2, deep comparison. For layers 0 to L−1 (L = `1` by default, capped by both models' depth), each of the seven projections `q, k, v, o, gate, up, down` of the suspect (`W_s`) and of the candidate (`W_c`) is compared:

- `raw_cos = cos(W_s, W_c)`, flattened, with a null: the mean cosine over `3` row shuffles of the candidate;
- `rel_d = ‖W_s − W_c‖_F / ‖W_c‖_F`;
- for `gate` and `up`, `nn_row`: for `512` sampled suspect neurons (rows), the best cosine against *any* candidate neuron, averaged. It does not care about neuron order, so it survives a permutation of the MLP. Its null shuffles the candidate's columns (`3` draws) and gives a z-score;
- `spectrum = 1 − ‖s_s − s_c‖ / ‖s_s‖`, the similarity of the singular values, after multiplying the inputs by the relevant norm gains. Singular values are invariant to rotation and permutation, but same-architecture models have similar spectra anyway, so this metric is weak and stays out of the verdict.

The Pearson correlation of each layer's norm gains is reported too. The embedding slice (1024 tokens starting at token 1000 by default) gets a raw cosine against a row-shuffled null.

Token circuits. To see through a rotation, the toolkit builds quantities in which `R` cancels. Starting from the embedding slice, RMS-normalised (`En = E / √(mean(E²) + ε)`, row by row, which a rotation preserves), with the layer-0 input norm gain folded into the attention inputs `A = En × g_in` and the post-attention gain into the MLP inputs `M = En × g_post`, four token × token matrices are computed:

```
emb_gram = En · Enᵀ
qk       = Σ_heads Q_h · K_hᵀ                    Q = A · W_qᵀ, K = A · W_kᵀ
ov       = (V · W_oᵀ) · Enᵀ                       V = A · W_vᵀ
mlp      = ((act(M · W_gateᵀ) × (M · W_upᵀ)) · W_downᵀ) · Enᵀ
```

Here · is a matrix product and × an element-wise one (a gain vector applied to every row, or the gated MLP's product). Rotary embeddings are the identity at position 0, so they are left out; key/value heads are repeated for grouped-query attention; biases are ignored (this is a fingerprint, not an exact forward pass). In each product, a rotation enters once as `R` and once as `Rᵀ` and cancels (`E·R · (W·R)ᵀ = E · Wᵀ`), and head or neuron permutations disappear in the sums. Each circuit of the suspect is compared with the candidate's by the Pearson r of their off-diagonal entries, against `10` nulls where the candidate's tokens are permuted consistently, giving a mean, a spread and a z-score. At least `64` tokens are needed. `emb_gram` is reported but excluded from the verdict: token semantics are much the same in every model, so the embedding Gram matrices of unrelated models correlate moderately.

All stage-2 randomness comes from one generator seeded with `0` and consumed in a fixed order, so the numbers are reproducible.

Verdict. Per candidate, the summary numbers are: the median `raw_cos` over every compared projection, the largest `rel_d`, the mean r and the smallest z of the `qk`, `ov` and `mlp` circuits, and `nn_row` of the layer-0 `up` projection with its z. The rules are checked in this order, the first that holds wins:

Table 11 — Provenance verdict ladder

Verdict	Condition
IDENTICAL weights	median raw_cos > 0.99999 and max rel_d < 1e-05
DERIVED, same basis	median raw_cos ≥ 0.8
RELATED	median raw_cos ≥ 0.3
DERIVED, transformed basis	circuits mean r ≥ 0.8 and min z ≥ 10
DERIVED, permuted MLP neurons	nn_row(up, layer 0) ≥ 0.8 and z ≥ 10
NO WEIGHT-LEVEL EVIDENCE	otherwise

IDENTICAL means a re-upload or a rename. DERIVED, same basis is a fine-tune or continued training. RELATED covers heavy continued training, merges and partial reuse: in validation, Qwen2.5-Coder against general Qwen2.5 (Coder is continued pre-training of it) scored a raw cosine of 0.36. DERIVED, transformed basis means the raw weights look unrelated but the invariant circuits match: a rotated or permuted copy. NO WEIGHT-LEVEL EVIDENCE means independent training, or distillation, which this test cannot detect.

Candidates are ranked by verdict, then raw cosine, then circuit r. The best match is named only if it reaches at least a transformed-basis verdict; the report warns when it differs from the declared base, or when a Hub suspect declares no lineage at all. The thresholds are heuristics, checked on real models without recalibration; the numbers behind a verdict should always be read against their nulls, and ideally against an unrelated candidate of the same architecture.

Behaviour probe

Weights say what changed; forward passes say what it does. The Behaviour Probe runs the same tokenized prompts through the base and the tuned model and compares their next-token distributions at every position.

KL divergence. At each position, with `p_t` and `p_b` the tuned and base next-token distributions over the shared vocabulary,

```
KL(tuned ‖ base) = Σ_tokens p_t · (log p_t − log p_b)
```

It is zero when the two models predict the same distribution and grows when the tuned model puts probability where the base did not. Weighting by `p_t` means the divergence is driven by what the *tuned* model expects. Per prompt, the mean and the maximum over positions are reported; the overall mean KL is the mean of the per-prompt means.

From the same forward passes come:

- the **hottest positions**: the positions with the largest KL, each with its context (the token and the ones before it, `9` tokens in all) and each model's top prediction there;
- the **next-token predictions** of both models after the prompt (top `5` with probabilities);
- the tokens **promoted** and **suppressed**: the mean of `log p_t − log p_b` per vocabulary token over every position of every prompt, largest gains and losses;
- the **hidden-state drift** per layer: the mean of `1 − cos(h_base, h_tuned)` over positions, entry 0 being the embeddings. It shows at which depth the two models' internal states part ways.

These statistics need token ids to mean the same thing in both models: when the two tokenizers' vocabularies differ, KL and token statistics are disabled and only generations run.

Control prompts. Any two models disagree somewhat on every prompt, so a raw KL says little. A fixed list of neutral everyday requests (a recipe, a capital, a short poem...) goes through the same comparison; their per-prompt mean KLs form the null distribution, and the target prompts' mean KL is reported as a z-score against it. A fine-tune aimed at a behaviour (a backdoor, a topic, an injected payload) shifts the prompts that exercise it more than neutral ones: a high z points at the probed behaviour. A z near 0 means the target prompts move no more than everyday text. At least two control prompts must succeed for the baseline to exist. Measuring a mean over many target prompts against the spread of single control prompts is conservative.

Generations. For the first prompts, both models generate a continuation greedily (always the most likely token), so differences come from the models, not from sampling. Generations are the most direct evidence: they show what the fine-tune actually writes.

Chat templates. By default each prompt is wrapped as a user turn of the chat template, with the assistant turn opened, using the tuned tokenizer's template if it has one, else the base's. Without any template, prompts are used as raw text and a warning says so. Inputs are truncated to `512` tokens. With a LoRA adapter only one model is in memory: the base outputs come from the same model with the adapter disabled.

Limitation for broad fine-tunes. The control z-score is meaningful for *targeted* fine-tunes. In validation, an adversarial fine-tune trained for many epochs changed general behaviour so much that the mean KL was about 4.7 on every prompt, controls included, and z stayed near 0 (−0.2 with the chat template, 0.6 without), even though the fine-tune did inject a payload. The evidence was elsewhere: a greedy generation for "Write a Python function to reverse a string" returned correct code plus an injected `sys.stdout = __import__('os').popen('calc.exe')`, and the hottest position showed the tuned model predicting "system" at the start of the prompt, matching the static Token Shifts finding. For broad fine-tunes, read the generations and the hottest positions, not the z-score.

Reading the evidence

Each analysis answers a narrower question than "is this model what it claims to be"; a conclusion comes from combining them, each read against its own baseline.

- **Origin.** Provenance tells whether the suspect's weights descend from a candidate, and in which way (same basis, related, transformed basis). Hub Lineage tells what the metadata claims. Agreement between the two is a confirmed lineage; a strong Provenance match with a different or missing declaration is the main finding of an audit.
- **Extent and location.** Weight Diff tells how much each tensor moved and where (modules, layers). It separates a LoRA (a few modules, exact zeros elsewhere), a full fine-tune (every trained tensor) and storage effects (every tensor, uniformly, within the quantization noise floor).
- **Structure.** Entropy Analysis tells whether the last layers' weight matrices were reshaped, not just displaced: a spectral-entropy shift beyond its random-update null means the fine-tune concentrated or spread the singular-value spectrum, a structural change a raw norm cannot show. It complements Weight Diff — a tensor can move a lot with little entropy change (a shift along existing directions) or little with a clear one (a change of rank) — and points at which layers specialised. A heavy Weight Diff move whose entropy shift stays within the null is displacement without restructuring.
- **Content.** Token Shifts names the tokens the heaviest updates promote, suppress or react to; the Behaviour Probe shows which predictions actually changed and what the model now writes. A token that appears both in a writer projection with a high z and among the tokens promoted or at the hottest

positions in the probe is a solid finding.

False positives to guard against:

- a raw change of a quantized tensor read as training: always check *beyond noise*;
- an entropy shift read as restructuring when it is within the random-update null (`|z|` small), or the entropy shift of a quantized tensor whose change is within the noise floor (a dequantization artefact);
- a token list from a direction at noise level (`|z|` small), or from a reader in a deep layer;
- `emb_gram` or a moderate circuit correlation read as kinship: unrelated models share token semantics. Compare with the null and with an unrelated same-architecture candidate;
- RELATED read as a fine-tune: a raw cosine between the RELATED and same-basis thresholds also fits continued pre-training of a common ancestor, or a merge;
- a high behaviour z driven by the target prompts simply being unusual text (very long, another language, code) rather than by a targeted change.

False negatives to keep in mind:

- distillation leaves no weight-level trace: NO WEIGHT-LEVEL EVIDENCE is not proof of independence;
- the true parent may not be among the candidates (search keeps root models by default, and only same-architecture ones): widen the search or give candidates explicitly;
- a broad fine-tune drowns a targeted change in the behaviour z-score: read the generations and hottest positions;
- prompts that never exercise a hidden behaviour (a backdoor trigger) show nothing: the static Token Shifts results, especially reader projections in early layers and moved embedding rows, can suggest which inputs to try;
- token lists depend on the base model's vocabulary matrices; an update that acts through combinations of features may not map cleanly onto any single token.

A sound report states the verdict, the numbers behind it with their nulls, and what each analysis could not have seen.

The suspicion assessment

The Analyse a Model workflow encodes the reasoning of *Reading the evidence* into one rule-based verdict, so a reader can trace every red flag back to the number that raised it. It establishes the base — a source given by the user (confirmed with Provenance) or the best weight-level parent it discovers — then runs the weight tools against it and turns each result into at most one **signal**, graded `none`, `low`, `medium` or `high` against that tool's own baseline:

- **Lineage.** A provided source the weights do not support, or a match only through a transformed or permuted basis, is `high`: a claim the weights contradict, or provenance laundering. A distant (RELATED) match is `medium`; a same-basis or identical match is `none`. With no source given, a discovered parent reached only through a transformed basis is `high`, and an undetermined origin is `medium` — and lowers the confidence, since the comparisons could not run.
- **Weight Diff.** Change concentrated in a few tensors (the heaviest moving far more than the median changed one) reads as targeted and scores `medium` to `high`; a broad change is `low`; a change within the quantization noise floor is `none`.

- **Entropy.** A spectral-entropy drop beyond the random-update null is restructuring: `medium`, or `high` well beyond it. Within the null, or a dequantization artefact on a quantized tensor, it is `none`.
- **Token Shifts.** A heavy update aligned with a small token set far beyond its token-shuffle null — possible trigger tokens — is `medium` to `high`; within the null it is `none`.
- **Vocabulary & Embeddings.** A reassigned token id, an added token that is both trained and emittable, or a heavy reader update triggered far beyond chance by an added token (the reader cross-check), is `high`: an alignment break (possible laundering), a token the model was taught to read and write (possible payload), or a planted trigger. A trained read-only token (the shape of a trigger) or a token-row movement concentrated far beyond the rest is `medium`; a vocabulary merely extended with untrained placeholders is `low`; no change is `none`.
- **Behaviour** (when run): prediction divergence beyond the control-prompt null is `medium` to `high`; within it, `none`.

The signals combine into the overall risk: any `high`, or two `medium`, makes it `high`; one `medium` is `elevated`; a `low` alone is `low`; none is `clean`. The confidence drops to `low` when the base could not be established or a tool failed, because the strongest evidence then never ran. The false-positive guards above apply unchanged — the quantization-noise and dequantization-artefact checks are built into the weight and entropy signals, and a within-null reading never raises a flag — so the verdict is only ever as strong as the baselines behind it. It automates the triage, not the judgement: the saved report carries every tool's full result so a reader can weigh the evidence directly.

Part 4 — Implementation

Shape of the code

LLM Forensics is one headless engine with two thin front-ends. The engine measures; the front-ends ask questions, start jobs, draw progress and show results. The engine never prints.

```
run.py                      run | web | check (front-ends imported lazily)
settings.py                 shared by both front-ends: __version__, report and models folders, HF_TOKEN
, defaults, limits, server address
progress.py                 Stage, ProgressEvent, ProgressSink, CancelToken, JobCancelled, quiet()
jobs.py                     JobRunner: one background job at a time
results.py                  ResultMeta, to_jsonable
environment.py              installed packages, GPU (nvidia-smi), available features
models/
  source.py                 TensorSource: names, shape, load(rows), config, quantization, aliases
  local.py hub.py           safetensors folders, local or on the Hub (HTTP Range)
  lora.py                   LoraAdapter, AdaptedSource (base + exact ΔW on the fly)
  gguf_source.py bnb.py     dequantizing readers for GGUF and bitsandbytes
  quant.py                  QuantScheme, NoiseFloor, noise_floor
  deltas.py                 iter_deltas: (W, ΔW) per tensor
  detect.py arch.py         detect, open_model, ModelRef, ModelKind, Architecture
  hub_search.py folders.py  Hub id search and local folder listing, for selecting a model
analysis/                   weights, shifts, vocab, entropy, lineage, provenance, behaviour; assessment
                            (the suspicion workflow that runs them and reads a verdict); stats, tokens
reporting/                  ReportStore, HTML rendering, Jinja2 templates (saved reports go to reports/
)
cli/                        rich + prompt_toolkit front-end; flows/ has one module per menu entry
web/                        Flask front-end; engine.py holds every decision and imports no Flask
```

Boundary. The engine (`models`, `analysis`, `reporting`, `progress.py`, `jobs.py`, `results.py`) never imports rich, prompt_toolkit or Flask. `run.py`, `environment.py`, `progress.py` and `settings.py` import only the standard library at module scope, so `python run.py check` works before torch is installed; torch, transformers, peft, gguf and huggingface_hub are imported inside the functions that need them. The modules sit at the project root (no package wrapper) and import as top-level names. What both front-ends decide (defaults, limits, the report and models folders, the Hub token from `HF_TOKEN`) lives in `settings`.

Reading models. `models.detect.detect` classifies a reference (local folder, `.gguf` file or Hub id) from file names and config only, and returns a `ModelRef` of kind full, adapter, GGUF or bitsandbytes. `models.detect.open_model` turns it into a `TensorSource` (`LocalSource`, `HubSource`, `AdaptedSource`, `GgufSource` or `BnbSource`) that returns one tensor, or a row range of it, as float32 on the CPU; the analyses never know which format they read. `models.deltas.iter_deltas` yields one `Delta` (base weight and update) per comparable tensor. For an adapter on its own base it visits only the adapted tensors and rebuilds ΔW exactly from the low-rank factors, which it keeps for a cheap exact SVD.

Jobs, progress and cancellation. `jobs.JobRunner.start` runs one analysis on a worker thread and returns a `Job` that accumulates progress events and then the result or the error. Only one job runs at a time, on purpose: two probes would compete for the same memory. An analysis receives a `ProgressSink` and calls `progress.report(sink, Stage.X, ...)`; the CLI (`cli.running.run_job`) draws the events as a progress bar, the browser polls them with a cursor (`Job.since`). Cancellation is cooperative: `CancelToken.check()` runs between units of work (tensors, candidates, pairs, prompts) and raises `JobCancelled`, so nothing stops mid-tensor. Hub lineage discovery and model loading for the Behaviour Probe are single library calls and cannot be stopped midway. `progress.quiet()` captures what the libraries print, so a failure can quote it.

Failures and results. One bad tensor, candidate, pair or prompt is recorded (a note, a warning, an `error` field) and the analysis continues; only `JobCancelled` and fatal input errors (`ModelError`, `UnsupportedModel`) propagate. Each analysis returns a dataclass tree of plain values (no tensors) with a `meta: ResultMeta` (analysis, inputs, parameters, date, tool version, notes, warnings). `results.to_jsonable` turns it into JSON data (non-finite floats become null); the JSON report is exactly that, and the HTML report and both front-ends render from it. After a successful job the front-end saves it with `reports.ReportStore.save`.

Libraries and functions

transformers

transformers is used only where a real tokenizer or model is needed. Weight Diff and Provenance never instantiate a model.

- `AutoTokenizer.from_pretrained` in `analysis.tokens.load_tokenizer`: Token Shifts decodes projected token ids with the base tokenizer, Hub Lineage with each parent's (`cli.flows.lineage.parent_decoder`, `web.engine.parent_decoder`), the Behaviour Probe loads both. A GGUF tokenizer is read from the file itself (`gguf_file=`); an adapter without tokenizer files falls back to its base.
- `AutoModelForCausalLM.from_pretrained` in `analysis.behaviour._load_model`, with `use_safetensors=True` for full models (pickle weights are never unpickled) and `gguf_file=` for GGUF models, which transformers dequantizes. The `dtype` keyword sets the precision.
- `AutoConfig.for_model` and `AutoModelForCausalLM.from_config` in `analysis.behaviour._load_bnb`: a bitsandbytes checkpoint is rebuilt as a plain model from its config (without `quantization_config`) and filled tensor by tensor from the toolkit's `BnbSource`.
- Chat templates in `analysis.behaviour._encode`: with the chat option on and a `chat_template` present, `apply_chat_template([{"role": "user", ...}], tokenize=False, add_generation_prompt=True)` wraps the prompt as a user turn (the tuned tokenizer's template first, `_template_owner`); inputs are truncated to `512` tokens.
- Forward passes in `analysis.behaviour._forward` with `output_hidden_states=True` under `torch.inference_mode()`, and greedy `model.generate(..., do_sample=False)` in `_generate_text`, so that runs are reproducible.
- Placement in `analysis.behaviour.load_pair`: models are loaded on the CPU, `plan_placement` decides from the weight sizes and `torch.cuda.mem_get_info()`, and each model is moved with `.to(device=..., dtype=...)`; `resolve_dtype` gives bfloat16 on a capable GPU, else float32. No `device_map` and no accelerate offloading: placement is explicit and reported.

peft

peft is used only by the Behaviour Probe, which needs a runnable adapted model. `PeftModel.from_pretrained(host_model, adapter_folder)` in `analysis.behaviour._load_adapted` loads the adapter, and `PeftModel.disable_adapter()` in `analysis.behaviour._with_base` gives the base outputs from the same model when the adapter is compared with its own base, so only one copy of the weights is in memory.

`merge_and_unload` is not used by the engine. Weight-level analyses apply an adapter themselves, exactly and lazily: `models.lora.LoraAdapter` reads `adapter_config.json` and the factors, `LoraAdapter.delta` rebuilds ΔW (rsLoRA scale included), and `models.lora.AdaptedSource` returns base plus ΔW tensor by tensor, as a merged model would, without writing it. DoRA and non-LoRA PEFT methods are refused there. The tests use peft's `get_peft_model` and `merge_and_unload` to build fixtures.

safetensors

`safe_open(path, framework="pt")` in `models.local.LocalSource` and `models.lora.LoraAdapter` indexes the keys of every file without loading data; `get_tensor` reads one tensor, `get_slice(name)[a:b]` only a row range, and `get_slice(name).get_shape()` a shape. This lazy, per-tensor access keeps memory to a few tensors at a time.

huggingface_hub

- `HfApi().list_repo_files` and `hf_hub_download` in `models.hub.HubRepo` list a repository and fetch small files (config, shard index, adapter files) and GGUF files into the HF cache.
- HTTP Range reads in `models.hub.HubRepo.read_range`, with `hf_hub_url`, `utils.build_hf_headers` (token) and the library's session `utils.get_session`. `HubSource._header` reads the first `8` bytes of a safetensors file (header length), then the JSON header (dtype, shape and byte offsets of every tensor); `HubSource.load` fetches only the bytes of the tensor or rows needed and rebuilds them with `torch.frombuffer`. A request is tried up to `4` times with a `120` s timeout; statuses `401, 403, 404` are not retried, and only 206 is accepted.
- Model info and the model tree: `HfApi.model_info(files_metadata=True)` gives tags, card data, config and file sizes (`analysis.lineage.discover_lineage`, `kind_of`, `size_gb`); the `base_model:{relation}:{repo}` tags are the Hub model tree, walked by `lineage.parents_of` and `provenance.declared_chain`. `HfApi.list_models(filter="base_model:{relation}:{repo}", sort="downloads")` lists children (`lineage.children_of`); with a `model_type` filter or `search=` it finds provenance candidates (`provenance.hub_search`).
- `scan_cache_dir` feeds the CLI completer with cached models (`cli.completer`); the same completer and the web pickers also suggest ids through `HfApi.list_models(search=)` (`models.hub_search.search_hub_models`), which returns nothing when the Hub is unreachable.

gguf

- `gguf.GGUFReader(path)` in `models.gguf_source.GgufSource` memory-maps the file: metadata through `reader.fields[key].contents()`, tensors through `reader.tensors`. Nothing is decoded before it is asked for.
- `gguf.quants.dequantize(data, tensor_type)` in `GgufSource.load` decodes one tensor or the rows asked for (a quantized row is a whole number of blocks, so raw rows are cut first).
- `gguf.MODEL_ARCH_NAMES` and `gguf.get_tensor_name_map(arch, blocks).mapping` in `models.gguf_source._reverse_name_map` map llama.cpp names (`blk.0.attn_q.weight`) back to every Hugging Face spelling, all kept as aliases (`TensorSource.aliases`).
- `gguf.GGMLQuantizationType` and `gguf.quants.quantize` in `models.quant._gguf_round_trip` simulate quantization round trips for exact noise floors.

torch and numpy

torch does all the arithmetic, float32 on the CPU for weights, with similarities accumulated in float64 (`analysis.stats.cosine`, `stats.pearson`). `torch.svd_lowrank` in `stats.leading_direction` is the randomized SVD of full updates, seeded inside `torch.random.fork_rng` so the result does not depend on visiting order; `torch.linalg.qr`, `svd` and `matrix_rank` in `stats._factored_svd` give the exact SVD and rank of a LoRA update through its thin factors; `torch.linalg.svdvals` serves `stats.spectrum_similarity`. `stats.seeded_generator` (`torch.Generator().manual_seed`) makes every null baseline reproducible. numpy only carries GGUF arrays between gguf and torch.

Other libraries, and what is not used

Jinja2 and MarkupSafe render the reports, rich and prompt_toolkit the CLI, Flask the web interface. **trl is not used**: the toolkit analyses fine-tunes but never trains anything. bitsandbytes is not used either: its formats are decoded in pure torch (`models.bnb`), and it is imported only by tests that check that decoding against bitsandbytes. accelerate is not used.

Analysis parameters

The terminal and the browser ask for the same parameters, with the same defaults and limits, taken from `settings`. Rows marked *engine only* are arguments of the analysis function that neither interface exposes; they keep their default. Every model field accepts a Hub id, a local folder or a `.gguf` file, and may ask for more:

Table 12 — Model field options (all analyses)

Option (UI)	Engine argument	Values	Default	Use
Model reference	`ref`	Hub id, folder, `.gguf` file	required	the model to read
Checkpoint	the chosen folder	a `checkpoint-N` of a training run	none	a run folder offers its checkpoints
GGUF file	`gguf_file`	a file of the folder or repository	most precise	asked when several GGUF files exist
Adapter base override	`base`, `suspect_base`, `tuned_base_override`	a full model	declared base	the model a LoRA adapter is applied to

Weight Diff

Measures, for every tensor present in both models, the relative change ‖ΔW‖/‖W‖, the cosine between W and W + ΔW, the largest absolute change, the fraction of elements that moved and, for LoRA updates, the exact rank. Engine function: `analysis.weights.compare_weights`.

Table 13 — Weight Diff parameters

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Base model	`base`	any model, usually full	required	—	reference weights	every tensor read	high
Tuned model	`tuned`	any model or adapter	required	—	compared weights	an adapter on its base reads only adapted tensors	high

There is no numeric parameter: a tensor counts as changed when ‖ΔW‖ ≥ `1e-06`, an element when its change exceeds the same value. Every tensor is read once, one at a time, so memory stays low and time grows with the model. An adapter compared with its own base is the fastest case. On the Hub every tensor is one Range request, so a whole-model comparison is latency-bound: use local copies for large models. When a side is quantized, each tensor also gets its noise floor (one quantization round trip more).

Token Shifts

Keeps the heaviest 2-D updates by leading singular value σ, projects each onto the vocabulary (writers through the unembedding, readers against the embeddings) against random directions, and lists the embedding rows that moved most. Engine function: `analysis.shifts.token_shifts`.

Table 14 — Token Shifts parameters

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Base model	`base`	any model, usually full	required	—	reference; gives tokenizer and vocabulary	every tensor read	high
Tuned model	`tuned`	any model or adapter	required	—	compared weights	as Weight Diff	high
Updates to project	`n`	whole number ≥ 1	`10`	`200`	updates kept and read as tokens	small	high
Tokens listed	`top`	whole number ≥ 1	`15`	`100`	tokens per axis end and per embedding matrix	negligible; size of the null statistic	medium
engine only	`rank`	whole number	6	—	rank of the randomized SVD of full updates	time per full tensor	low
engine only	`null_draws`	whole number	`20`	—	random directions per null	precision of z	low

Every changed tensor goes through an SVD whatever `n` is, so the cost is reading the weights and the randomized SVD of full updates, not the number of updates kept. Raise `n` for full fine-tunes, where many tensors move by similar amounts; LoRA updates are usually dominated by a few tensors and directions. The null compares the mean of the top `top` scores with the same statistic for random directions, so a larger `top` averages over weaker tokens and lowers z. Read |z| below 3 as noise.

Vocabulary & Embeddings

Diffs the two tokenizers (tokens added, removed or reassigned to a different id, changed special tokens and normalizer), compares the embedding rows that map to the same token in both (the shared rows) one token at a time, characterises each added token from its embedding and unembedding rows, and — when tokens were added — re-projects the heaviest reader updates onto the suspect's extended vocabulary so a trigger that is a new token can surface (the reader cross-check Token Shifts cannot do, since it reads against the base vocabulary). Engine function: `analysis.vocab.compare_vocab`; the tokenizer vocabulary comes from `analysis.tokens.vocab_info`. Fills the gap the shape-matched tools leave when the vocabulary grows, since they drop the embedding and the unembedding as unmatched.

Table 15 — Vocabulary & Embeddings parameters

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Base model	`base`	any model, usually full	required	—	reference and its tokenizer	embedding and unembedding read	high
Tuned model	`tuned`	any model or adapter	required	—	compared model and its tokenizer	embedding and unembedding read	high
Moved rows listed	`top_rows`	whole number ≥ 1	`20`	`200`	shared rows listed per matrix	negligible	medium
New tokens read	`new_tokens`	whole number ≥ 1	`50`	`1000`	added tokens characterised	one embedding row each	medium
engine only	`reader_updates`	whole number	`3`	—	heaviest reader updates re-projected onto the suspect vocabulary (0 off; only when tokens were added)	a second weight pass	medium
engine only	`neighbours`	whole number	`8`	—	nearest existing tokens per new token	negligible	low
engine only	`rank`	whole number	`6`	—	rank of the randomized SVD of a full reader update	time per reader tensor	low

A reassigned token (same text, a different id) breaks the id-to-id alignment the shared-row comparison relies on, so it is reported on its own and only the truly aligned rows are compared. A new token counts as *trained* when its embedding norm is at least 0.5 of the mean existing-row norm, and *emittable* when the untied unembedding carries a comparable row; a tied model emits whatever it was trained to read. Quantized embeddings are read against their noise floor, so a dequantization artefact is not reported as a move.

Hub Lineage

Reads a model's Hub metadata (declared parents and children), keeps the comparable pairs, then runs Token Shifts on each chosen pair, reading both models by HTTP Range. Engine functions: `analysis.lineage.discover_lineage`, then `analysis.lineage.analyse_lineage`.

Table 16 — Hub Lineage parameters

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Model	`model_id`	a Hub id	required	—	the model investigated	—	high
Mode	`mode`	`auto`, `parent`, `children`	auto	—	parents, children or both	children multiply the pairs	high
Maximum variants	`max_variants`	whole number ≥ 1	`5`	`50`	children kept, most downloaded first	one pair per child	medium
Maximum size (GB)	`max_size_gb`	number > 0	`8`	none	larger full and GGUF children skipped	bounds downloads and time	medium
Pairs to analyse	`pairs`	all, or chosen pairs	all	pairs found	pairs compared	time per pair	high
engine only	`n`, `top`	whole numbers	10, 12	—	as Token Shifts, per pair	as Token Shifts	low
engine only	`rank`, `null_draws`	whole numbers	6, `20`	—	as Token Shifts	as Token Shifts	low

Parents come from the `base_model:{relation}:{repo}` tags; relations `finetune, adapter, quantized` are compared; a merge is compared with its first comparable parent (else its first listed one). Children are listed for the relations `adapter, finetune, quantized`. Adapters are never skipped for size; a GGUF child is measured by the file that would be read and is downloaded whole. Discovery is cheap and can be repeated; the time goes into the pairs, each a full Token Shifts run over the network. Start with mode *parent* to check a declared base, then *children* with few variants, and lower the size limit on a slow connection.

Provenance

Screens candidate parents cheaply on layer 0, compares the best in depth on the first layers and an embedding slice (raw cosine, relative distance, neuron matching, spectra and token circuits, each against a shuffled null), and gives a verdict per candidate. Engine function: `analysis.provenance.find_provenance`.

Table 17 — Provenance parameters

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Suspect model	`suspect`	any model, adapter, GGUF or bitsandbytes	required	—	the model investigated	read partially	high
Candidate parents	`candidates`	references, comma or line separated	none	web: `50`	explicit candidates, always deep-compared	one deep comparison each	high
Search the Hub	`search`	yes / no	yes	—	add Hub models of the same `model_type`	many screens, network	high

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Search text	`search_text`	free text	empty	—	replaces the `model_type` filter	changes the pool	medium
Include models that declare a base	`include_derived`	yes / no	no	—	keep non-root search hits	larger pool	medium
Layers compared in depth	`layers`	whole number ≥ 1	`1`	`64`	layers 0 to layers − 1	time linear in layers	medium
Top k	`top_k`	whole number ≥ 1	`3`	`40`	screened candidates sent to the deep stage	one deep comparison each	high
engine only	`search_limit`	whole number	40	—	search hits kept	screening time	low
engine only	`deep_all`	yes / no	no	—	deep-compare every screened candidate	time	low
engine only	`slice_rows`	whole number	64	—	rows per tensor in the screen	screen precision	low
engine only	`emb_start`, `emb_rows`	whole numbers	1000, 1024	vocabulary	embedding slice and circuit tokens	memory quadratic in rows	low
engine only	`workers`	whole number	6	—	threads for checks and screening	network parallelism	low

Explicit and declared candidates (the base an adapter declares, and `base_model` tags walked up `4` generations) are always compared in depth, on top of the `top_k` best search hits. Candidates whose hidden size, MLP size, head counts or head dimension differ are skipped; layer count and vocabulary size may differ. The defaults are the recommended setting: layer 0 and the embeddings move least during fine-tuning, so they are the strongest fingerprint, and each extra layer costs the matrices `q, k, v, o, gate, up, down` per candidate, over the network for Hub candidates. Raise `layers` to see how far a derivative diverges with depth; raise `top_k`, or name the parent explicitly, when the search pool is large. Search keeps root models only by default: an Instruct model declares a base and is excluded unless derived models are included. The circuits need at least `64` tokens in the slice. Verdicts use fixed thresholds:

Table 18 — Provenance verdict thresholds

Verdict	Condition
IDENTICAL	median raw cosine > `0.99999` and max rel_d < `1e-05`
DERIVED, same basis	median raw cosine ≥ `0.8`
RELATED	median raw cosine ≥ `0.3`
DERIVED, transformed basis	circuits mean r ≥ `0.8` and min z ≥ `10`
DERIVED, permuted neurons	layer-0 nn_row ≥ `0.8` and z ≥ `10`

The null draws are fixed too: `3` row shuffles per tensor, `3` column shuffles per neuron comparison over `512` sampled neurons, `10` token permutations per circuit, all from one generator seeded with 0.

Behaviour Probe

Runs prompts through both models: KL(tuned ‖ base) per position, hottest positions, next-token predictions, promoted and suppressed tokens, hidden-state drift per layer, a z-score against neutral control prompts, and optional generations. Engine functions: `analysis.behaviour.probe` (batch), `load_pair`, `compare_prompt` and `generate` (interactive session).

Table 19 — Behaviour Probe parameters

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
Base model	`base`	full, GGUF or bitsandbytes	required	—	reference model	loaded in memory	high
Tuned model	`tuned`	any model or adapter	required	—	compared model	an adapter on its base shares one copy	high
Mode	—	batch, interactive	batch	—	prompts file, or typed prompts	interactive keeps the models loaded	high
Prompts file	path or upload	`.txt`, `.json`, `.jsonl`	required (batch)	web: `2,097,152` bytes	target prompts	time linear	high
JSON field	`field`	field name	`question`	—	prompt field of JSON objects	none	medium
Maximum prompts	`limit`	whole number ≥ 1	`50`	`10,000`	prompts read from the file	time linear	medium
Wrap prompts in the chat template	`chat`	yes / no	yes	—	prompt as a user turn	changes what is measured	high
Tokens generated	`generate_tokens`	whole number ≥ 0	0 (none)	`1024`	greedy generations by both models	slowest part	high
Positions and tokens listed	`top`	whole number ≥ 1	`15`	web: `100`	hottest positions, promoted and suppressed tokens	negligible	low

Parameter (UI)	Argument	Values	Default	Limit	Purpose	Impact	Relevance
engine only	`generate_prompts`	whole number	`3`	—	first prompts that get generations	time	low
engine only	`controls`	prompts	`CONTROL_PROMPTS`	—	null baseline of KL	fixed extra cost	low
engine only	`device`, `dtype`	auto, cpu, cuda; auto, float32, bfloat16, float16	auto	—	placement and precision	memory, speed	low

`top` is set in the interactive session (`:top` in the terminal, a field in the browser) and used when the session is saved; a batch run lists `15`. Prompts are truncated to `512` tokens. Automatic placement puts both models on the GPU when their weights fit in `0.85` of its free memory, else the tuned model on the GPU and the base on the CPU, else both on the CPU; on a GPU they run in bfloat16 when supported. When the tokenizers differ, KL and token statistics are disabled and only generations run (`64` tokens when none were asked).

Keep the chat template on for instruction-tuned models, which is how they are used; turn it off for base models and raw-text triggers. Use prompts that exercise the suspected behaviour: the control z-score is high only when the fine-tune changed *these* prompts more than everyday ones. For broad fine-tunes z stays near 0 whatever the prompts, and the evidence is in the generations and the hottest positions, so ask for generated tokens. Each prompt costs one forward pass per model and each generated token one more: on a CPU, prefer small models and few prompts.

Quantization parameters

GGUF files and bitsandbytes checkpoints are read dequantized to float32, one tensor at a time, so every analysis accepts them. A comparison with a quantized model mixes what fine-tuning changed and what quantization destroyed; the second is measured per tensor as a **noise floor**.

Table 20 — Quantized and refused formats

Input	Reader	Status
GGUF, any type the gguf package can dequantize	`models.gguf_source.GgufSource`	supported; the Behaviour Probe loads it through transformers
bitsandbytes `nf4`, `fp4` (4-bit) and int8	`models.bnb.BnbSource`	supported, without bitsandbytes
any other `quant_method` (GPTQ, AWQ, EXL2...)	`models.detect._classify`	refused: only `bitsandbytes` and GGUF
pickle weights `.bin`, `.pt`, `.pth`, `.ckpt`	`models.detect._classify`	refused: unpickling can run code
DoRA, non-LoRA PEFT methods	`models.lora`	refused

A tensor whose GGUF type cannot be dequantized fails alone and is noted; Hub Lineage skips repositories with another quantization method and says why.

GGUF file choice

A folder or repository holding several GGUF files asks which one to read, listing them from the most precise. Unattended reads (Hub Lineage children, Provenance candidates, any choice left on `auto`) take the most precise. Precision is read from the type in the file name, in this order, unknown types last: `F32, BF16, F16, Q8_0, Q6_K, Q5_K_M, Q5_K_S, Q5_K, Q5_1, Q5_0, Q4_K_M, Q4_K_S, Q4_K, Q4_1, Q4_0, IQ4_XS, IQ4_NL, Q3_K_L, Q3_K_M, Q3_K_S, IQ3_S, IQ3_XXS, Q2_K, IQ2_S, IQ2_XS, IQ2_XXS, IQ1_M, IQ1_S`. A GGUF file on the Hub is downloaded whole into the HF cache; safetensors are read by Range.

Q/K permutation in llama-type GGUF

llama.cpp's `convert_hf_to_gguf.py` permutes the rows of `q_proj` and `k_proj` for the architectures `deci, granite, granitemoe, llama`, interleaving the two rotary halves of every head. `GgufSource.load` reverses it with `unpermute_rows`, using the `attention.head_count` metadata for Q and `attention.head_count_kv` for K, so tensors line up with the original weights; a row-range read of such a tensor dequantizes it whole first. Names are mapped back through the gguf package's table and a config is rebuilt from the metadata.

bitsandbytes layout

A 4-bit weight is stored as packed codes (two per byte, high nibble first), one `absmax` scale per block, a 16-value code book (`quant_map`) and a JSON state (shape, block size, nesting). With double quantization the scales are themselves quantized (`nested_absmax`, `nested_quant_map`, offset) and decoded first. The block size comes from the state, with `64` as fallback. An int8 weight has one scale per row (`SCB`): W = weight × SCB / `127`. The result equals bitsandbytes' own dequantization.

Noise floors

`models.quant.noise_floor` quantizes the **base** tensor with the compared tensor's scheme and dequantizes it again: the relative error ‖Q(W) − W‖ / ‖W‖ is the change the tensor would show had fine-tuning not touched it. It is measured on `256` evenly spaced rows (all rows of a smaller tensor), because a relative error converges quickly.

Table 21 — Noise floor by scheme

Scheme	Floor	Method
`F32, F64`	0	lossless
GGUF types with a Python quantizer (legacy, ternary, F16, BF16)	exact	`gguf.quants.quantize` then `dequantize`
GGUF K- and IQ-quants, other types without one	estimated, labelled	uniform asymmetric quantizer on blocks of 32
bitsandbytes nf4, fp4	exact	block-wise absmax, nearest value of the file's code book
bitsandbytes int8	exact	row-wise absmax over ±127 levels

Bit widths used for estimates: `Q2_K: 2, Q3_K: 3, Q4_K: 4, Q5_K: 5, Q6_K: 6, Q8_K: 8, IQ1_S: 1, IQ1_M: 1, IQ2_XXS: 2, IQ2_XS: 2, IQ2_S: 2, IQ3_XXS: 3, IQ3_S: 3, IQ4_NL: 4, IQ4_XS: 4, NVFP4: 4`; a type missing from this list is estimated at 4 bits. K-quants are somewhat more accurate than the uniform model, so the estimate errs on the high (conservative) side. The double quantization of bitsandbytes scales is not simulated.

`analysis.stats.quantization_floor` combines the floors of both sides in quadrature when both are quantized, and gives no floor to an adapter compared with its own base (its update is exact). `analysis.stats.beyond_noise` calls a change real when it exceeds `2` floors **and** `1e-05`, the rounding level; the second condition matters for F16 and BF16, whose floors are tiny.

Table 22 — Where floors are used

Analysis	Use
Weight Diff	per tensor: floor, exact or estimated, beyond-noise flag; count beyond noise
Token Shifts, Hub Lineage	per kept update: floor and beyond-noise flag, with a warning
Provenance	none: quantization lowers raw cosines only slightly, thresholds still apply
Behaviour Probe	none: the models are compared by their outputs

Never read a raw change of a quantized tensor on its own: a model merged into a 4-bit base and saved again differs on every tensor by about its floor.

Reports

Every successful analysis is saved by `reports.ReportStore.save` in the report directory (`settings.reports_root`), which is the `reports` folder of the project (`settings.default_reports_root`) until the Settings menu stores another:

```
reports/<id>/report.json   to_jsonable(result): the complete data
reports/<id>/report.html   rendered from report.json, self-contained
reports/<id>/meta.json     sidecar: id, analysis, title, dates, inputs, one-line summary
```

The id is `DATE-TIME-ANALYSIS-SLUG`, the local time of the save formatted as `%Y%m%d-%H%M%S`. The slug comes from the title or the inputs (last component of local paths, Hub ids whole, tuned model first), reduced to lower-case letters, digits and dashes, at most `40` characters; a numeric suffix avoids collisions. Files are written atomically and the sidecar last, so a folder without it is incomplete and not listed. The JSON is strict (no NaN) and keeps non-ASCII text. Ids from URLs are checked against a strict pattern before any file access, and the folder must resolve inside the reports folder.

`reporting.html.render_html` renders from the JSON form, never from live objects, so `ReportStore.rerender` can rebuild a page after a template change. The templates in `reporting/templates` are `base.html.j2` (header, inputs, parameters, notes, warnings), `macros.html.j2` (stat cards, bars, token chips, null comparisons), `styles.css.j2` (inline stylesheet, light print override), one template per analysis (`weights: weights.html.j2, shifts: shifts.html.j2, vocab: vocab.html.j2, entropy: entropy.html.j2, lineage: lineage.html.j2, provenance: provenance.html.j2, behaviour: behaviour.html.j2, assessment: assessment.html.j2`), `shifts_sections.html.j2` shared by Token Shifts and Hub Lineage, `vocab_sections.html.j2` shared by Vocabulary & Embeddings and the assessment addendum, and `generic.html.j2` as fallback. Pages carry no script and no external resource, and autoescaping is always on: token texts, prompts and generations are untrusted. The Weight Diff page lists the `50` largest changes (the JSON has all), and scores with $|z| \geq 3$ are flagged.

The Reports menu lists, opens, re-renders and deletes reports; the browser also downloads HTML and JSON, and shows a report under a content policy that allows no script.

Tests and validation

Before any change is declared done, `venv/Scripts/python.exe -m pytest`, `venv/Scripts/ruff.exe format .`, `venv/Scripts/ruff.exe check .` and `venv/Scripts/mypy.exe run.py settings.py environment.py progress.py jobs.py results.py analysis cli models reporting web tests` must all pass.

Table 23 — Test layout

Files	Cover
`test_models_*.py`	detection, local and Hub sources, LoRA, GGUF and bitsandbytes readers
`test_analysis_*.py`	each analysis on synthetic models, quantized comparisons, golden tests
`test_reports.py`	saving, listing, rendering and escaping of reports
`test_cli_*.py`	flows driven by scripted answers (`cli_support.py`)
`test_web_*.py`	API, pages and engine through the Flask test client (`web_support.py`)
`test_foundation.py`, `test_jobs.py`, `test_progress.py`, `test_settings.py`, `test_environment.py`, `test_run.py`	engine plumbing, entry point
`test_hub_network.py`	live Hub checks, opt-in

Synthetic models. Tests never touch the network. `tests/conftest.py` builds one tree of models per session (fixture `models`): a base, a sharded copy, a LoRA adapter, the merged model, a training run with checkpoints and a full checkpoint. `tests/helpers.py` writes safetensors folders, adapters, GGUF files laid out as llama.cpp's converter does (Q/K rows permuted) and bitsandbytes checkpoints; its `FolderRepo` serves a folder as a fake Hub repository, so the Range reader runs on disk. `tests/behaviour_fixtures.py` builds tiny transformers models with a tokenizer, a peft adapter, the merged model and, when bitsandbytes is installed, an NF4 copy.

Golden tests. Ported computations are compared with the reference scripts the toolkit was derived from (weight comparison, token shifts, provenance, the last bit for bit). A port or behaviour change of such a computation gets a golden test.

Opt-in and conditional tests. `tests/test_hub_network.py` runs only with `LLMFORENSICS_NETWORK=1`; it checks that tensors read by Range equal those of the downloaded file on the real Hub. The GPU placement test needs CUDA, and the tests comparing the decoder with bitsandbytes skip without it. pytest runs with `-p no:faulthandler`: on Windows, MKL raises access violations it handles itself. The web interface is tested with the Flask test client only, never in a browser.

Real-model validation. `validation/validate_real_models.py` runs each analysis on real checkpoints with a known ground truth (LoRA fine-tunes with their merged and GGUF exports, base models from the HF cache, purpose-built controls). Experiments can be selected by name (for example `gguf bnb`), paths are overridden by environment variables, and the numbers go to a JSON results file. They are recorded in `docs/validation.md`: adapter versus merged agreement, GGUF name mapping and Q/K un-permutation, measured versus estimated floors, bitsandbytes decoding, provenance calibration, Hub lineage, Token Shifts on a real adapter, and a Behaviour Probe that exposes an adversarial payload.